CHAPTER 10

MLOps: Packaging, Serving, Tracking and Continuous Delivery

Course week 10–11

Learning Objectives

After working through this chapter you will be able to:

- Define the deployable unit of a machine learning system as a *decision function*, and package it so that preprocessing, threshold and metadata travel with the weights.
- Choose a serving pattern from latency, freshness and cost constraints, and size a service with a latency budget and Little's law.
- Build a typed, tested scoring API with FastAPI, and explain which endpoints must be synchronous and which asynchronous.
- Containerise the service in a small, reproducible, non-root Docker image, and reason about image size on constrained networks.
- Explain why a model is reproducible only when code, data, configuration, environment and seed are all recorded, and record them with MLflow.
- Govern releases through a model registry with aliases, so that promotion and rollback are single, auditable pointer moves.
- Test data, model behaviour, business cost and fairness as code, with statistically honest gates.
- Automate testing, training, building and deployment with GitHub Actions, keeping a human approval where regulation requires one.

10.1 Introduction: from notebook to production system

Every chapter so far has ended at the same place: a model that scores well on a held-out set. That is where the machine learning course of a decade ago ended too, and it is where most projects in industry quietly die. The notebook contains a model; nobody else can call it; it cannot say which data produced it; and the one person who knows how to rerun it is on leave.

Sculley et al. (2015) made the point with a picture that has become the emblem of the field, redrawn in Figure 10.1. In a production machine learning system, the code that learns is a small box surrounded by much larger ones: configuration, data collection and verification, feature extraction, serving infrastructure, monitoring, testing. The small box is what Chapters 1 to 9 taught. This chapter builds three of the large boxes around it, serving, testing and continuous delivery, and Chapter 11 builds the fourth.

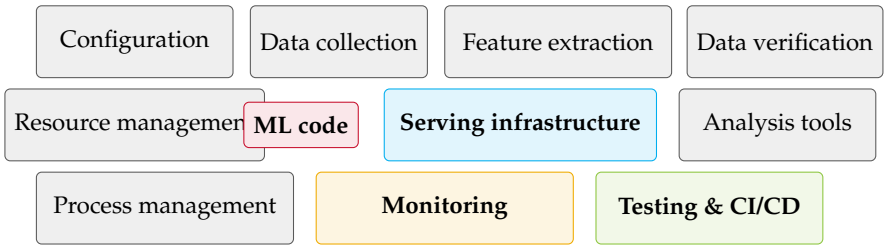


Figure 10.1: After Sculley et al. (2015). The learning code is the smallest box. This chapter builds the cyan and green boxes; Chapter 11 builds the amber one.

What actually changes when a model leaves the notebook? Not the weights. What changes is the environment around them, and Table 10.1 lists the differences that matter. Each row is a requirement that no amount of additional model accuracy can satisfy, and each row maps to a section of this chapter.

	In the notebook	**In production**
Inputs	A clean `DataFrame` you prepared	Untrusted JSON from strangers, at any hour (Section 10.3.3)
Execution	Once, when you press run	Continuously, for thousands of callers (Section 10.3)
Latency	Irrelevant	A contract, stated as a percentile
Identity	`model_final_v3.keras`	An immutable, traceable version (Sections 10.4.1–10.4.3)
Failure	Re-run the cell	A borrower is harmed, an auditor asks why (Section 10.6.5)

Table 10.1: Same weights, different world. Every row is a requirement that a better model cannot meet.

Case Study

PesaCred, round ten: shipping the thin-file scorer PesaCred, the Nairobi digital lender that has accompanied us since Chapter 1, lends to borrowers with no bureau history, using mobile-money behaviour alone. Its default rate is about $\pi = 4\%$, and its costs are asymmetric: a missed default costs roughly KES 10,000 in lost principal, while wrongly declining a good borrower costs roughly KES 800 in lost margin and goodwill.
The credit model of Chapters 3 to 6 is now to be offered over a USSD menu on basic feature phones. A borrower dials a short code, answers two questions, and the telco's USSD gateway calls PesaCred's scoring service with a JSON request. The service must answer before the menu step times out; the product team has allocated two seconds per step end to end. Three further requirements come from outside engineering:

1. The Central Bank of Kenya's Digital Credit Providers Regulations 2022 require that customer complaints can be investigated, which in practice means any decision must be reconstructable months later.
2. The Data Protection Act 2019 requires data minimisation (Section 25) and gives borrowers rights around decisions made solely by automated processing (Section 35).
3. The risk committee wants every model change approved by a named risk officer, with the evidence attached.

The first half of this chapter turns the model into a service that meets the two-second budget. The second half turns the service into a system that can be changed safely every week, and that can answer the auditor.

10.1.1 The MLOps lifecycle and maturity levels

MLOps is the discipline of running machine learning systems in production reliably and repeatedly. It borrows heavily from DevOps, the practice of automating software delivery, and adds what DevOps never needed: data that changes without a commit, models whose quality is statistical rather than binary, and the need to retrain as the world drifts (Kreuzberger et al., 2023).

The shape to keep in mind is a loop rather than a line (Figure 10.2). Data feeds training; the trained model is packaged and served; its behaviour is tracked and tested; the whole path is automated; and monitoring of the live model eventually triggers new data collection and retraining. A deployment is not the end of a project but one turn of the loop.

It helps to know where an organisation stands on this loop. Google's

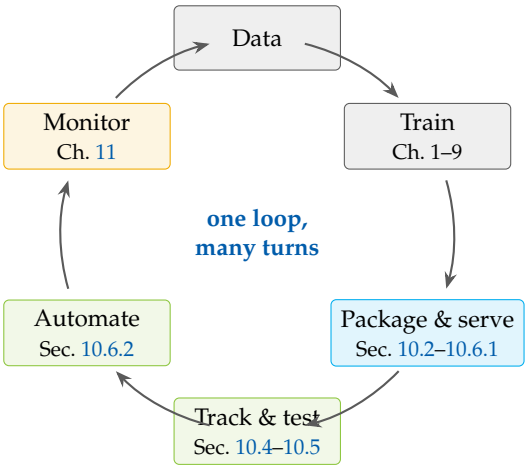


Figure 10.2: The MLOps lifecycle, with the part of the book that covers each stage.

framing of MLOps maturity (Google Cloud, 2020), sketched in Figure 10.3, distinguishes three levels. At **level 0** a person trains, exports and deploys by hand, and retraining is an event. At **level 1** retraining runs as an automated pipeline, which is called *continuous training*. At **level 2** the pipeline itself is tested and deployed by continuous integration and delivery, so that changing how models are trained is as safe as changing an application. Sections 10.2 and 10.3 take PesaCred to a disciplined level 0; Sections 10.4 to 10.6 take it to levels 1 and 2.

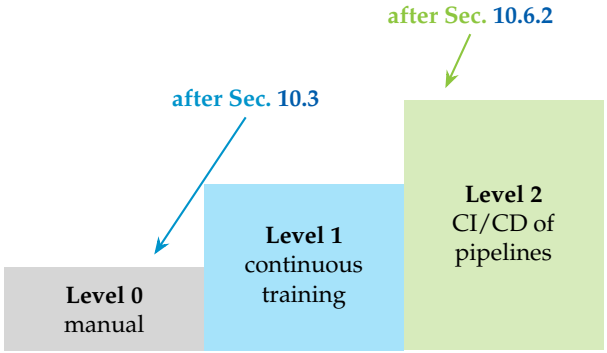


Figure 10.3: Three levels of MLOps maturity. Each step removes a human from a repetitive path, never from an accountable decision.

10.2 Packaging the model artefact

10.2.1 What exactly is "the model"?

Ask a data scientist to hand over the model and you will usually receive a file of weights. A file of weights cannot make a decision. To turn a borrower's raw fields into an approve-or-decline, the service also needs the preprocessing that the weights expect, the threshold that turns a probability into an action, a statement of what inputs are valid, and a record of where all of it came from.

Definition 10.1 (Decision function and model artefact)**.** The *decision function* of a deployed classifier is

$$\delta(\boldsymbol{x}) \;=\; \mathbb{1}\big[\, f_{\boldsymbol{\theta}}\big(\phi(\boldsymbol{x})\big) \geq t^{\star} \big]\,, \tag{10.1}$$

where ϕ is the preprocessing map, $f_{\boldsymbol{\theta}}$ the trained model with parameters $\boldsymbol{\theta}$, and $t^{\star}$ the decision threshold. The *model artefact* is the versioned bundle containing $\boldsymbol{\theta}$, ϕ, $t^{\star}$, the input and output schema, and the lineage metadata needed to reproduce it.

The definition has an uncomfortable consequence, drawn in Figure 10.4: change *any* layer of the bundle and you have a new model version, even if the weights are untouched. Re-fitting the normaliser on fresher data is a new model. Moving the threshold because the cost of a default changed is a new model. Treating these as configuration tweaks that bypass versioning is how organisations end up unable to say which decision rule was applied to a customer last March.

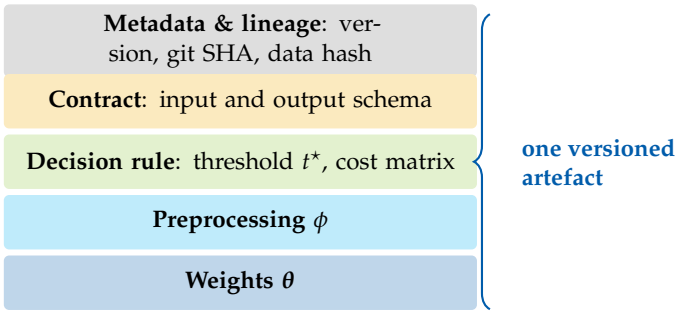


Figure 10.4: Anatomy of a model artefact. All five layers travel together, or none should.

10.2.2 Training–serving skew

The most common silent failure of deployed models is not a bug in the model but a disagreement about ϕ. During development, preprocessing lives in the notebook, written in pandas. When the model is deployed, an engineer rewrites the same logic in the serving code, in a hurry, from a

description. The two implementations then differ in some small way: a logarithm in a different base, a missing clip on outliers, a region code that appeared after training, a mean imputed from production data rather than from the training set.

Definition 10.2 (Training–serving skew)**.** *Training–serving skew* occurs when the preprocessing applied at serving time differs from the one applied during training, $\phi_{\text{serve}} \neq \phi_{\text{train}}$, so that f_θ receives inputs from a distribution it never saw.

What makes skew dangerous is that it passes every offline test (Figure 10.5). Validation used ϕ_{train}, so the AUC in the report is genuine. The degradation appears only in production, where it is easily mistaken for data drift, the subject of Chapter 11, and investigated in the wrong place for weeks.

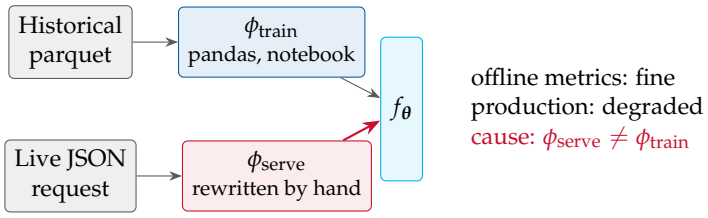


Figure 10.5: Training–serving skew. The model is correct; the two preprocessing paths disagree.

The cure is structural rather than procedural: do not write ϕ twice. Keras preprocessing layers make this natural, because statistics computed by `adapt()` become part of the model graph and are saved with the weights. Listing 10.1 builds the thin-file scorer this way. The serving code then passes *raw* fields, and there is no second implementation of ϕ to drift.

```
import keras
from keras import layers

NUM = ["tenure_months", "avg_monthly_inflow_kes", "txn_count_90d", "
    ontime_repayments_12m"]
inputs = {c: keras.Input(shape=(1,), name=c) for c in NUM}
inputs["region"] = keras.Input(shape=(1,), dtype="string", name="region")

norm = {c: layers.Normalization(name=f"norm_{c}") for c in NUM}
for c in NUM:
    norm[c].adapt(train_df[[c]].to_numpy())            # mean and variance become
    weights
region = layers.StringLookup(output_mode="one_hot", num_oov_indices=1)
region.adapt(train_df["region"].to_numpy())            # unseen codes map to an OOV
    bucket

x = layers.Concatenate()([norm[c](inputs[c]) for c in NUM] + [region(inputs["region"
    ])])
x = layers.Dense(32, activation="relu")(x)
pd_out = layers.Dense(1, activation="sigmoid", name="pd")(x)
```

```
model = keras.Model(inputs, pd_out, name="thinfile_scorer")
```

Listing 10.1: Preprocessing inside the Keras graph. Normalisation statistics and the region vocabulary are frozen into the artefact.

Two details deserve a comment. The `num_oov_indices=1` argument decides in advance what happens when a region code the model never saw arrives in production: it is mapped to a dedicated bucket rather than crashing the service or, worse, being silently mapped to a real region. And the heavily skewed inflow feature would normally be log-transformed first; that transform belongs in the graph too, for exactly the same reason, rather than in a pre-processing script that the serving team might forget.

10.2.3 Serialisation formats

Saving the artefact is a choice between fidelity to the training framework and portability across runtimes (Figure 10.6). The native `.keras` format preserves everything, including custom layers and the preprocessing graph, but needs Python and Keras to load; it is the right choice for the FastAPI service in this chapter. The TensorFlow **SavedModel** format is the input to TF Serving and to TFLite conversion. **ONNX** is runtime-neutral and useful when the serving stack is not Python, at the cost of checking that every operation is supported. **TFLite** targets phones and the tablets that agents carry in areas without reliable signal. And **pickle** deserves its own warning.

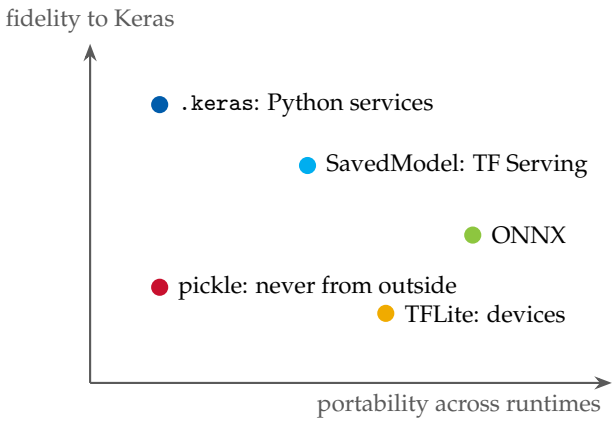


Figure 10.6: Serialisation formats trade fidelity against portability. Pickle is low on both and unsafe besides.

Pitfall

Pickle executes code Loading a pickle file runs arbitrary Python chosen by whoever wrote the file. Accepting a pickled model from a vendor, a public repository or a colleague's laptop is equivalent to running their code on your production server. Keras 3's loader defaults to `safe_mode=True`, which refuses to deserialise arbitrary `Lambda` code for the same reason; prefer built-in layers over lambdas so that you never need to switch it off.

A serialisation you have not reloaded is a serialisation you have not tested. Listing 10.2 saves both formats and then asserts that the reloaded model reproduces the original's predictions on real validation rows. That assertion costs a second to run and becomes a unit test in Section 10.5.

```
import numpy as np, keras

model.save("artefacts/scorer.keras")           # Keras-native, used by the API
model.export("artefacts/scorer_savedmodel")    # SavedModel, for TF Serving / TFLite

reloaded = keras.saving.load_model("artefacts/scorer.keras")   # safe_mode=True
probe = {k: v[:256] for k, v in X_val.items()}                 # real validation rows
np.testing.assert_allclose(model(probe), reloaded(probe), atol=1e-6)
```

Listing 10.2: Save, reload, and prove the round trip.

10.2.4 The threshold belongs in the package

Chapter 3 argued that a decision threshold should come from costs rather than from habit. Packaging makes the argument concrete, because the threshold is the one number that converts the model's output into a borrower's outcome.

Proposition 10.1 (Cost-optimal threshold)**.** *Let $p = \Pr(\textit{default} \mid \boldsymbol{x})$ be a calibrated probability, and let C_{FN} be the cost of approving a defaulter and C_{FP} the cost of declining a good borrower. The decision that minimises expected cost is to decline exactly when*

$$p \;>\; t^{\star} \;=\; \frac{C_{FP}}{C_{FP} + C_{FN}}. \tag{10.2}$$

Proof. Approving costs C_{FN} with probability p, so its expected cost is $p\,C_{FN}$. Declining costs C_{FP} with probability $1 - p$, so its expected cost is $(1 - p)\,C_{FP}$. Declining is cheaper when $p\,C_{FN} > (1 - p)\,C_{FP}$, which rearranges to $p > C_{FP}/(C_{FP} + C_{FN})$. □

For PesaCred, $t^{\star} = 800/10{,}800 \approx 0.074$: decline anyone whose probability of default exceeds about seven per cent. The number is far from the 0.5 that a library's `predict` method would use, and the difference is money.

Figure 10.7 plots expected cost per applicant against the threshold for a calibrated scorer: the minimum sits at $t^\star$, and using 0.5 instead costs roughly 60% more per applicant.

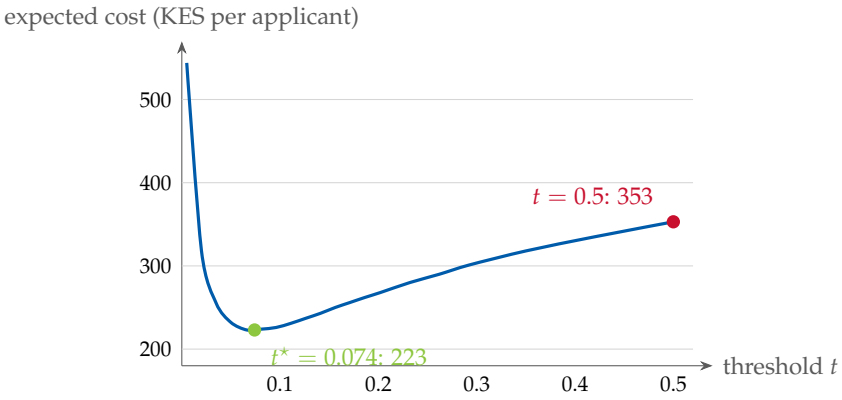


Figure 10.7: Expected cost against threshold for a calibrated scorer (simulated, illustrative). The minimum is where Proposition 10.1 puts it.

Two conditions travel with the proposition. It holds only if p is *calibrated*, which is why the calibration check of Chapter 3 must be repeated on every candidate model, not once. And it depends on a cost matrix that the business owns and may change, so the cost matrix belongs in the artefact next to $t^\star$, where a reviewer can see that the two are consistent.

10.2.5 Metadata and environment

The last two layers of Figure 10.4 answer the questions an auditor asks first: which code, which data, which threshold? A small metadata file written at training time answers them for anyone who holds the artefact folder (Listing 10.3). It is deliberately simple; in Section 10.4.1 MLflow formalises exactly this record.

```
import json, subprocess, hashlib

def sha256(path):
    return hashlib.sha256(open(path, "rb").read()).hexdigest()

meta = {
    "model_name": "thinfile-scorer", "version": "1.3.0",
    "git_sha": subprocess.check_output(["git", "rev-parse", "--short", "HEAD"]).
     decode().strip(),
    "data_sha256": sha256("data/snapshots/2026-09-30.parquet"),
    "features": NUM + ["region"],
    "threshold": 0.074, "cost_matrix_kes": {"fn": 10000, "fp": 800},
    "val_metrics": {"auc": 0.81, "cost_kes": 223},        # [PLACEHOLDER]
    "keras_version": keras.version(),
}
json.dump(meta, open("artefacts/metadata.json", "w"), indent=2)
```

Listing 10.3: A metadata sidecar written next to the weights.

The environment is part of the model too. A model trained in September under one set of library versions may refuse to load, or load and give slightly different numbers, under the versions installed in March (Figure 10.8). The remedy is to declare direct dependencies in a short `requirements.in`, compile them into an exact, hashed lock file, and install only from the lock everywhere: laptop, CI runner and container.

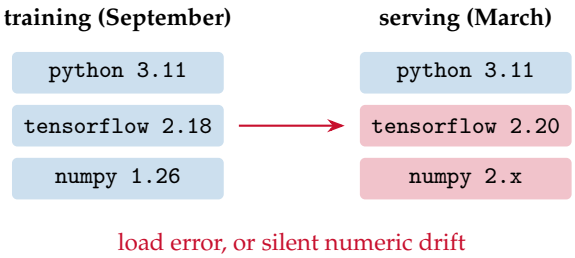


Figure 10.8: Dependency drift. The fix is a hashed lock file installed identically everywhere.

```
uv pip compile requirements.in --generate-hashes -o requirements.lock
pip install --require-hashes -r requirements.lock
```

Listing 10.4: Compile and install an exact, hashed lock file.

10.3 Serving the model: design and API

10.3.1 Four ways to serve predictions

"Deploy the model" hides a design decision that should be made first: when, and where, does a prediction get computed? Four patterns cover almost every system (Figure 10.9). In **batch** serving, a scheduled job scores every customer and writes the results to a table; PesaCred's nightly refresh of credit limits is batch. In **online** serving, a request arrives and must be answered within milliseconds; the USSD loan application is online. **Streaming** serving scores events as they flow past, as fraud detection on mobile-money transfers does. **Edge** serving runs the model on the device itself, which is the only option for an agent's tablet with no signal.

Real systems are usually hybrids, and PesaCred's is a good example (Figure 10.10). The expensive features, ninety-day aggregates over millions of mobile-money transactions, change slowly, so a nightly batch job computes them and writes them to a key-value feature store indexed by the hashed phone number. At request time the online service looks them up in

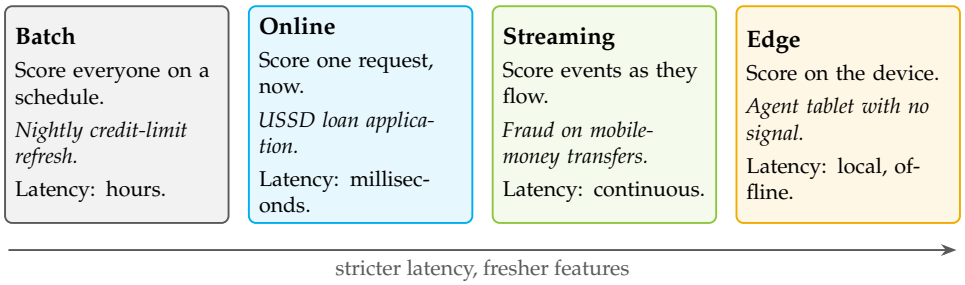


Figure 10.9: Four serving patterns, ordered by how fresh the features and how fast the answer must be.

a few milliseconds and combines them with the fields in the request. The general rule is worth stating as a principle: move every computation that does not depend on the request out of the request path.

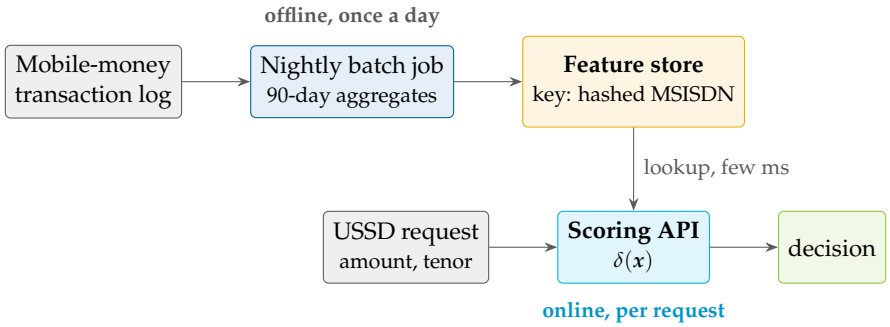


Figure 10.10: PesaCred's hybrid design: heavy aggregates in batch, the decision online.

10.3.2 Latency, capacity and model placement

A model that is accurate but slow is not deployable, so the next three questions are quantitative: how much time does the model get, how many workers does the service need, and where should the model run relative to the API.

10.3.2.1 A latency budget

Before writing any service code, decompose the service-level agreement into its parts. The end-to-end time a borrower experiences is a sum,

$$T_{\text{e2e}} \;=\; T_{\text{net}} + T_{\text{feat}} + T_{\text{inf}} + T_{\text{post}} \;\le\; T_{\text{SLA}}, \tag{10.3}$$

of network and gateway time, feature lookup, inference, and post-processing including logging. Figure 10.11 shows PesaCred's allocation of its two-

second USSD step. The model receives 150 milliseconds. That single number settles several design questions before they are asked: it rules out a multi-billion-parameter language model in the decision path, it makes a small dense network or a gradient-boosted ensemble comfortable, and it leaves 900 milliseconds of headroom for retries and for the long tail discussed next.

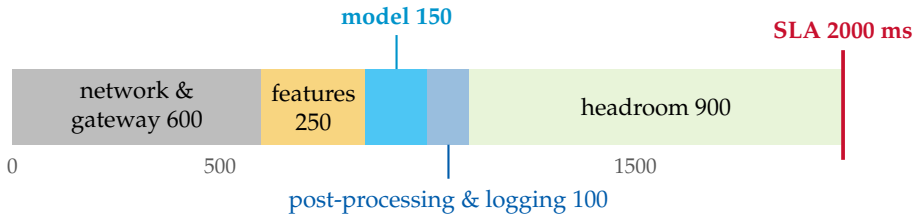


Figure 10.11: A latency budget for one USSD menu step (design allocation, in milliseconds).

10.3.2.2 Tail latency is what users feel

Latency is a distribution, not a number, and its shape is almost always skewed to the right (Figure 10.12). A service whose average is 40 milliseconds may still take over 100 for one request in a hundred. Averages hide exactly the requests that time out, so budgets and service-level agreements are stated as percentiles: p_{50}, p_{95}, p_{99}.

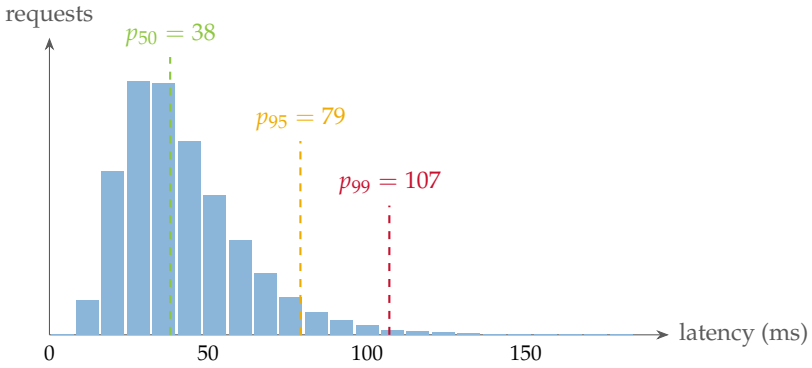


Figure 10.12: A typical right-skewed latency distribution (simulated). The budget is set against the tail, not the centre.

The tail matters more than intuition suggests because modern requests fan out. PesaCred's scorer calls a feature store, a fraud check and a logging service, and each call is an independent draw from its own latency distribution.

Proposition 10.2 (Fan-out amplifies the tail)**.** *If a request makes n independent calls, each of which finishes below its own p_q latency with probability q, the probability that none of them is slow is q^n.*

The proof is one line of independence, and the consequence is sobering. With $q = 0.99$ and $n = 5$, $0.99^5 \approx 0.951$: one request in twenty hits at least one component's p_{99}, so the *system*'s p_{95} is set by its components' p_{99} (Dean and Barroso, 2013). This is why the budget in Figure 10.11 keeps headroom, and why every dependency in the request path must itself have a percentile target.

10.3.2.3 How many workers? Little's law

Capacity planning needs one result from queueing theory, and it is remarkably general. **Little's law** states that in any stable system the average number of requests in flight equals the arrival rate times the average time each spends in the system,

$$L = \lambda W. \tag{10.4}$$

It holds regardless of the arrival pattern or the service-time distribution, which is what makes it useful before a service exists. Applied to workers, it gives the *offered load* $a = \lambda \bar{S}$, the average number of workers kept busy by arrivals at rate λ with mean service time $\bar{S}$. To keep each worker's utilisation below a safe ceiling $\rho_{\max}$,

$$c \geq \left\lceil \frac{a}{\rho_{\max}} \right\rceil = \left\lceil \frac{\lambda \bar{S}}{\rho_{\max}} \right\rceil. \tag{10.5}$$

At PesaCred's typical peak of $\lambda = 120$ requests per second with $\bar{S} = 25$ milliseconds, the offered load is $a = 3$ workers' worth of work, and a ceiling of $\rho_{\max} = 0.6$ gives $c = 5$ workers. The ceiling is not timidity. Queueing delay grows without bound as utilisation approaches one, so a service run at 95% utilisation has a beautiful average and a catastrophic tail. Month-end, when salaries land and loan demand spikes, is where this bites (Exercise 10.3).

10.3.2.4 Model-in-service or model-as-a-service?

The last serving decision is where the model runs relative to the API (Figure 10.14). The simplest design loads the model inside the web service process; one image, one deployment, and model and API scale together. The alternative runs the model in a dedicated model server such as TF Serving or Triton, called over gRPC by a thin API that handles validation and business rules. The dedicated server buys independent scaling, GPU batching and multi-model hosting, at the price of one more network hop and one more thing to operate. PesaCred's small CPU model is a textbook case for

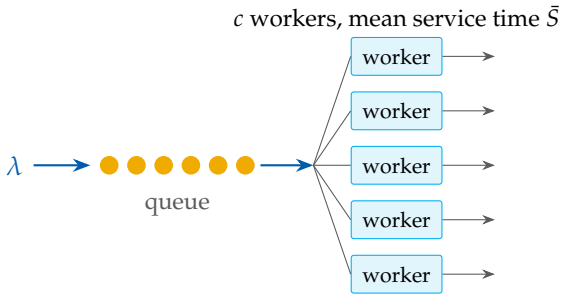


Figure 10.13: A multi-worker service. Little's law needs only λ and the time in the system.

the embedded design, which is what the rest of this chapter builds; the calculus reverses for the GPU-hungry language models of Section 10.3.5.

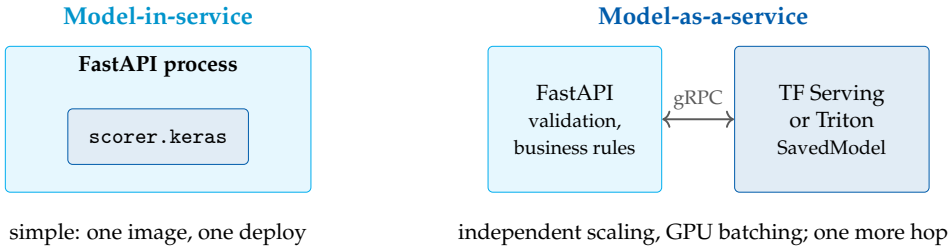


Figure 10.14: Two placements of the model relative to the API.

10.3.3 The FastAPI service

10.3.3.1 Why FastAPI

FastAPI is a Python web framework built on two ideas that suit machine learning services unusually well (Ramírez, 2024). The first is that Python type hints, interpreted by the Pydantic library, become request validation, response serialisation and machine-readable documentation at once (Figure 10.15). The second is that it runs on ASGI, the asynchronous server interface, so synchronous and asynchronous endpoints can live side by side. You write a schema once and it becomes the contract for callers, for tests, and, as Section 10.3.5 shows, for language-model agents.

10.3.3.2 Contract first

Write the request schema before the endpoint. Listing 10.5 describes a valid applicant, and every constraint in it encodes domain knowledge: tenure cannot be negative or exceed forty years, the region must be one of eight known codes, and unknown fields are rejected rather than silently ignored.

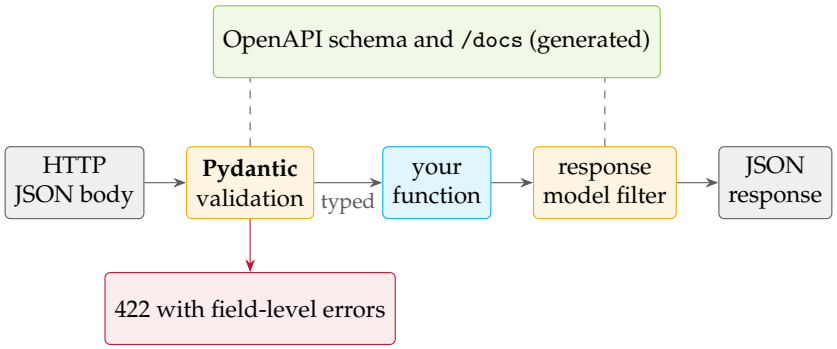


Figure 10.15: The FastAPI request path. Invalid input never reaches your code, and the schemas also generate the documentation.

Invalid requests receive a 422 response with a field-level explanation and never reach the model.

```python
from typing import Literal
from pydantic import BaseModel, ConfigDict, Field

Region = Literal["nairobi", "central", "coast", "eastern",
                 "north_eastern", "nyanza", "rift_valley", "western"]

class Applicant(BaseModel):
    model_config = ConfigDict(extra="forbid")            # unknown fields -> 422
    msisdn_hash: str = Field(pattern=r"^[0-9a-f]{64}$")  # salted SHA-256, never the
      number
    tenure_months: int = Field(ge=0, le=480)
    avg_monthly_inflow_kes: float = Field(ge=0, le=5_000_000)
    txn_count_90d: int = Field(ge=0)
    ontime_repayments_12m: int = Field(ge=0, le=365)
    region: Region
```

Listing 10.5: The request schema. Bounds are the first defence against garbage in.

The response deserves as much care as the request, because it is the audit record of a credit decision. Every field of Listing 10.6 answers a question someone will eventually ask: the customer asks why, the risk officer asks which threshold, the engineer asks which model, and the auditor asks to see this exact call. The probability is optional because some cases are deliberately *referred* to a human rather than scored, for example an applicant with no transaction history at all.

```python
class ScoreResponse(BaseModel):
    request_id: str                                  # joins logs and complaints
    pd: float | None = Field(default=None, ge=0.0, le=1.0)   # None when referred
    decision: Literal["approve", "refer", "decline"]
    threshold: float                                 # t* used for this call
    reason_codes: list[str]                          # top drivers of the score
    model_version: str                               # e.g. "1.3.0+a1b2c3d"
```

Listing 10.6: The response schema: a decision together with its provenance.

Kenya Context

Data minimisation under Section 25 of the Data Protection Act is implemented in the schema itself: the service receives a salted hash of the phone number, which is enough to look up features and join logs, and never the number. Section 35's safeguards on solely automated decisions are implemented by the `refer` outcome, which routes a case to a human loan officer. Both are design features of the contract, not paragraphs in a policy document.

10.3.3.3 Loading the model once

Loading a Keras model takes seconds; scoring one applicant takes milliseconds. A service that loads the model inside the request handler therefore spends almost all its time loading (Figure 10.16). FastAPI's *lifespan* handler runs code once at startup, before the first request, and once at shutdown (Listing 10.7).

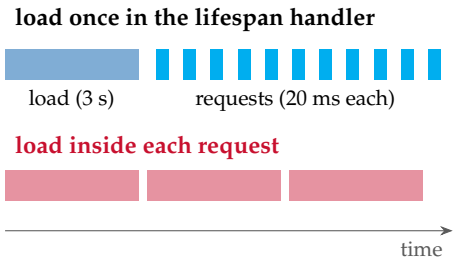


Figure 10.16: Loading per request turns a 20-millisecond service into a three-second one.

```
import json
from contextlib import asynccontextmanager
import keras
from fastapi import FastAPI

STATE: dict = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    STATE["model"] = keras.saving.load_model("artefacts/scorer.keras")
    STATE["meta"] = json.load(open("artefacts/metadata.json"))
    yield                                   # the application serves requests here
    STATE.clear()                           # graceful shutdown

app = FastAPI(title="Thin-file scorer", version="1.3.0", lifespan=lifespan)
```

Listing 10.7: Load the artefact once, at startup.

10.3.3.4 The scoring endpoint

With the contract and the loaded artefact in place, the endpoint itself is thin (Listing 10.8): validate, apply business rules, predict, apply the threshold, explain. Two lines deserve attention. The business rule that refers applicants with no history runs *before* the model, so the model is never asked a question it has no basis to answer. And the model is called directly rather than through `predict()`, which is designed for large batches and adds noticeable overhead for a single row.

```
import numpy as np
from uuid import uuid4

@app.post("/v1/score", response_model=ScoreResponse)
def score(a: Applicant) -> ScoreResponse:                # def, not async def
    meta, rid = STATE["meta"], str(uuid4())
    feats = a.model_dump(exclude={"msisdn_hash"})
    base = dict(request_id=rid, threshold=meta["threshold"], model_version=meta["
     version"])

    if feats["txn_count_90d"] == 0:                        # business rule before the
      model
        return ScoreResponse(**base, decision="refer", reason_codes=["NO_HISTORY"])

    x = {k: np.array([[v]]) for k, v in feats.items()}
    pd_ = float(STATE["model"](x, training=False)[0, 0])   # direct call, no predict
     ()
    decision = "decline" if pd_ >= meta["threshold"] else "approve"
    return ScoreResponse(**base, pd=pd_, decision=decision,
                         reason_codes=top_reasons(x))      # attribution helper
```

Listing 10.8: The scoring endpoint.

10.3.3.5 `def` or `async def`?

The endpoint is declared with `def`, not `async def`, and the choice is not stylistic. An `async def` function runs *on* the event loop, the single thread that accepts and dispatches every request. If such a function performs CPU-bound work, as a TensorFlow forward pass is, the loop is blocked and every other request waits behind it (Figure 10.17). A plain `def` function is run by FastAPI in a thread pool, leaving the loop free. The rule is therefore simple: `async def` for input and output, such as database queries, HTTP calls and streamed language-model tokens; `def` for computation. Getting it wrong produces a service that passes every single-user test and collapses in the first load test.

A related point concerns processes and threads. Uvicorn can run several worker processes, each with its own copy of the model, and TensorFlow can use several threads per process. Their product must not exceed the number of CPU cores, or the workers fight over the same cores and every request slows down. On a four-core machine, four workers with one

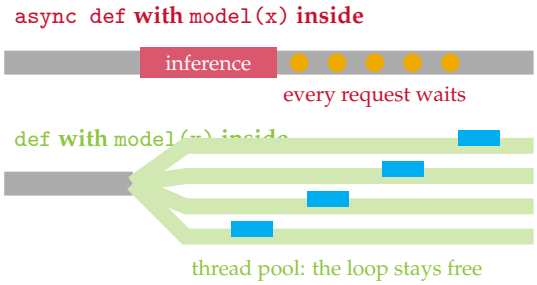


Figure 10.17: CPU-bound inference inside `async def` blocks the event loop; inside `def` it runs in a thread pool.

intra-op thread each is a sensible default, and memory then scales as the number of workers times the model size.

10.3.4 Operating the service: health, errors and tests

An orchestrator such as Docker or Kubernetes needs to ask a running service two different questions. *Liveness*: is the process alive, or should it be restarted? *Readiness*: has it finished loading the model, so that traffic may be sent to it? A container that is still loading a model is alive but not ready, and conflating the two either kills slow starters or routes requests to services that cannot answer. Two tiny endpoints, `/health` and `/ready`, plus a `/metadata` endpoint that returns the artefact's provenance, cover the need.

Errors are part of the contract too (Table 10.2). Each status code tells the caller what to do next, and stack traces never leave the service; they are logged against the `request_id`. For a credit product one rule matters above the others: a timeout or internal error must fail *closed*. The USSD menu says "we will send you an SMS", and the case is queued for review. It never approves by default.

Code	Meaning	What the caller should do
200	Scored	Show the decision
422	Invalid input	Fix the request; the body lists the offending fields
503	Not ready	Retry with back-off; the model is still loading
500	Internal error	Stop; the service pages its on-call engineer

Table 10.2: Status codes as instructions to the caller.

Batch callers, such as the nightly limit refresh, should not call the online endpoint once per customer. A model call on a batch of b rows costs roughly $T(b) = \alpha + \beta b$, where α is a fixed dispatch overhead and β the

marginal cost per row, so the cost per row

$$\frac{T(b)}{b} = \frac{\alpha}{b} + \beta \tag{10.6}$$

falls towards β as the batch grows (Figure 10.18). A separate `/v1/score/batch` endpoint accepting a list of applicants captures that saving.

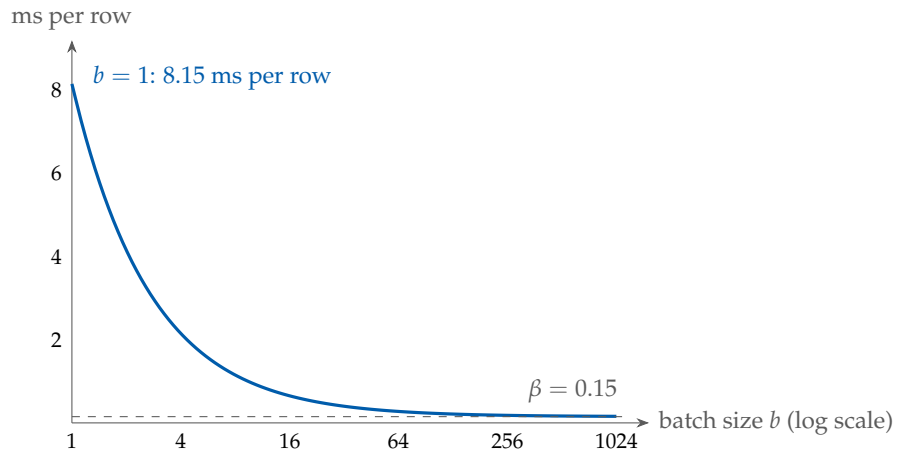


Figure 10.18: Batching amortises the fixed cost of a model call (illustrative $\alpha = 8$ ms, $\beta = 0.15$ ms).

Finally, test the service the way a caller would. FastAPI's `TestClient` runs the lifespan handler, the validation and the endpoint in-process (Listing 10.9). These tests take seconds and become the merge gate of the pipeline in Section 10.6.2.

```
from fastapi.testclient import TestClient
from app.main import app

GOOD = {"msisdn_hash": "a" * 64, "tenure_months": 26, "avg_monthly_inflow_kes": 18
    _500.0,
       "txn_count_90d": 143, "ontime_repayments_12m": 9, "region": "nairobi"}

def test_score_contract():
    with TestClient(app) as client:                    # runs the lifespan: model
     loads
        r = client.post("/v1/score", json=GOOD)
        assert r.status_code == 200
        assert 0.0 <= r.json()["pd"] <= 1.0
        assert r.json()["decision"] in {"approve", "refer", "decline"}

def test_rejects_negative_tenure():
    with TestClient(app) as client:
        assert client.post("/v1/score", json={**GOOD, "tenure_months": -3}).
     status_code == 422
```

Listing 10.9: Contract tests for the scoring API.

10.3.5 Serving language-model and agent workloads

Chapter 9 produced two new kinds of artefact, a fine-tuned adapter and a retrieval index, and PesaCred wants to serve them alongside the scorer. The packaging discipline of this chapter carries over unchanged; the physics does not (Table 10.3).

	Tabular scorer	Language-model service
Latency	10–50 ms	1–20 s
Hardware	CPU, about 100 MB	GPU, tens of GB
Output	One JSON object	A stream of tokens
Unit cost	Negligible	Per token
Determinism	Exact	Sampled and prompt-sensitive

Table 10.3: The same packaging discipline, different operating characteristics.

The first consequence is architectural: language models stay *out* of the two-second decision path. They belong where latency is tolerated, such as drafting an explanation for a loan officer, triaging documents or composing collections messages. The second is that their endpoints stream. A response of several hundred tokens takes seconds to generate, and sending tokens as they are produced, using Server-Sent Events, makes the wait tolerable. Here `async def` is the right choice, because the service spends its time waiting on the model server rather than computing (Listing 10.10).

```
from fastapi.responses import StreamingResponse

class ExplainRequest(BaseModel):
    request_id: str                          # the scoring call being explained
    language: Literal["en", "sw"] = "en"     # English or Kiswahili

@app.post("/v1/explain")
async def explain(req: ExplainRequest):
    record = await decision_log.get(req.request_id)       # reason codes, no raw PII
    async def events():
        async for chunk in llm.stream(build_prompt(record, req.language)):
            yield f"data: {chunk}\n\n"                     # Server-Sent Events
     framing
    return StreamingResponse(events(), media_type="text/event-stream")
```

Listing 10.10: A streaming explanation endpoint. The language model verbalises reason codes the scorer already produced.

The division of labour in that listing is the governance point. The language model explains a decision that the audited scorer has already made; it does not make one. The same principle shapes the integration with agents. An underwriting agent built on a language model (Chapter 9) should obtain a probability of default by *calling* the scoring service as a tool, never by estimating one itself (Figure 10.19). Conve-

niently, the Pydantic contract of Listing 10.5 already is a tool definition: `Applicant.model_json_schema()` produces the JSON Schema that agent frameworks expect, so the API, its tests and the agent can never disagree about what a valid applicant is.

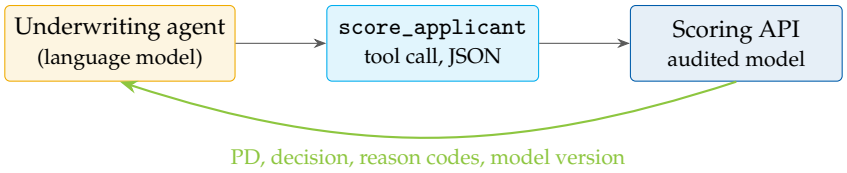


Figure 10.19: The agent orchestrates; the audited model decides. The tool schema is the API's own Pydantic contract.

10.4 Reproducibility, experiment tracking and the registry

Section 10.3 ends with a working, tested service, and with three questions it cannot answer. Why this model, and not one of the forty others trained last month? Could we rebuild it exactly, six months from now, for a CBK review? And could we ship a better one next week without a hero working through the night? This section and the two that follow answer them in turn, and the first answer begins with a precise statement of what a trained model depends on.

A trained model is the output of a procedure $\mathcal{T}$ applied to five inputs,

$$M \;=\; \mathcal{T}\big(\text{code},\ \mathcal{D},\ \lambda,\ \text{env},\ s\big), \tag{10.7}$$

namely the code at a specific commit, the training data $\mathcal{D}$, the configuration and hyperparameters λ, the software environment, and the random seed s (Figure 10.20). Change any one and you obtain a different model. Most teams record λ carefully, because hyperparameters feel like the interesting part, and forget $\mathcal{D}$ and the environment, which is where irreproducibility actually comes from. *Lineage* is simply a stored value for every argument of Equation (10.7).

Even with all five recorded, some variation remains. Weight initialisation, shuffling and dropout are random; some GPU kernels accumulate floating-point sums in a nondeterministic order; and a SQL query over live tables returns different rows next month. The last source is removed by training only on immutable, hashed snapshots, never on a query. The other sources are controlled in two lines,

```
keras.utils.set_random_seed(42)                  # Python, NumPy and TF seeds
tf.config.experimental.enable_op_determinism()   # deterministic GPU kernels
```

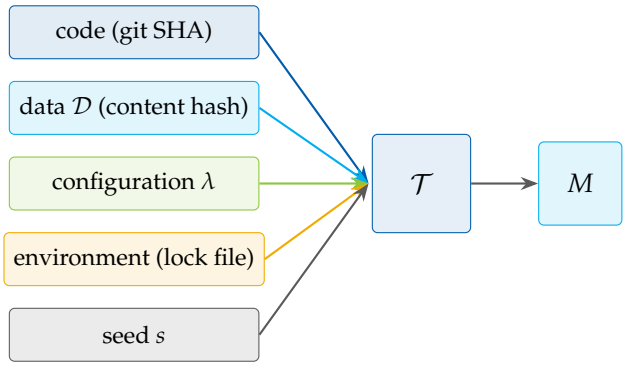


Figure 10.20: A model is a function of five inputs. Lineage records all five.

the second at some cost in speed, which is worth paying for an audit rebuild and not for every tuning trial.

The more important lesson is statistical. Because training is random, a single run is a single sample. Figure 10.21 shows validation AUC for two configurations over twenty seeds each. Their seed-to-seed spread is wider than the difference between their means, so comparing one run of each is comparing noise. The honest summary is an interval: with k seeds of mean $\bar{m}$ and standard deviation s,

$$\bar{m} \pm t_{k-1,\,0.975}\,\frac{s}{\sqrt{k}}. \tag{10.8}$$

Here configuration A gives 0.807 [0.805, 0.809] and B gives 0.808 [0.804, 0.811]. The intervals overlap almost entirely; B's luckiest seed, 0.820, against A's unluckiest, 0.795, would look like a breakthrough and is nothing of the kind.

10.4.1 Experiment tracking with MLflow

Spreadsheets and folders of files named `model_final_REAL.keras` fail for the same reason: they rely on memory, and memory does not scale to forty experiments, three people and six months. *Experiment tracking* replaces memory with a queryable record. Huyen (2022) draws the useful distinction: tracking answers "what did we try, and how did it do?", while versioning answers "can we recreate it?". A good tracking system does both.

Definition 10.3 (Tracked run)**.** A *run* is the record of one execution of a training procedure. It holds *parameters* (set once), *metrics* (time series indexed by step), *artefacts* (files, including the model), *tags* (searchable labels, including the git SHA and the data hash), and system information such as start time, duration and host.

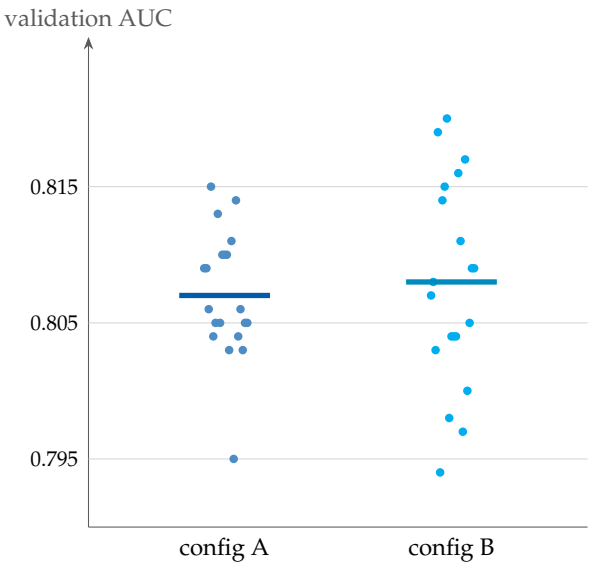


Figure 10.21: Twenty seeds per configuration (simulated). Compare distributions, not single runs.

MLflow is the open-source tracking platform used in this book (Zaharia et al., 2018). It is framework-agnostic and self-hostable, and it has four components that matter here: **Tracking** stores runs; **Models** packages trained models in a standard format with a signature; the **Registry** gives models names, versions and aliases (Section 10.4.3); and **Tracing** records spans for language-model and agent calls (Section 10.4.2).

10.4.1.1 Architecture

A self-hosted tracking server separates two very different kinds of data (Figure 10.22). The *backend store*, a SQL database such as PostgreSQL, holds small structured records: runs, parameters, metrics and registry entries. The *artefact store*, an object store speaking the S3 protocol, holds the large files: model weights, plots, evaluation reports and logged samples. Notebooks, CI runners and the serving API all talk to the same server, which is what makes a model trained in a notebook and a model trained by the pipeline comparable in one table.

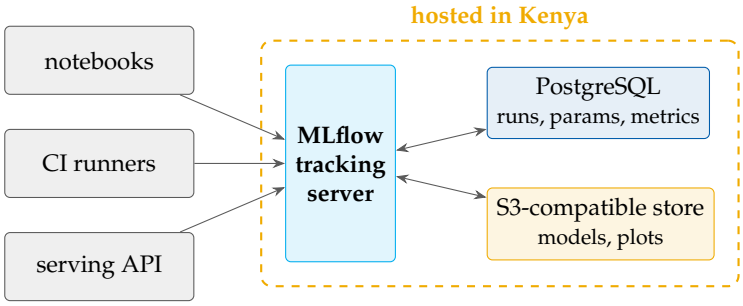


Figure 10.22: An MLflow tracking server with separate metadata and artefact stores, kept in-country.

Kenya Context

Logged artefacts can contain personal data: evaluation samples, error analyses, even input examples stored with a model. Keeping both stores in-country simplifies the cross-border transfer obligations of Part VI of the Data Protection Act 2019. Any S3-compatible object store will do. MinIO was for years the default choice for self-hosting, but its community edition entered maintenance mode in late 2025, with new features reserved for a commercial edition; teams starting now should also evaluate actively maintained alternatives such as Garage, SeaweedFS or Ceph's object gateway. MLflow is indifferent to the choice, since it speaks only the S3 protocol.

10.4.1.2 Logging runs

Listing 10.11 instruments a training run explicitly. Note what is logged besides AUC: the business metric, expected cost at $t^\star$, which is the number a promotion decision will actually use; the p_{99} latency, which the budget of Section 10.3.2.1 constrains; and the git SHA and data hash as tags, which are the lineage.

```
import mlflow

mlflow.set_tracking_uri("http://mlflow.ilab.internal:5000")
mlflow.set_experiment("thinfile-scorer")

with mlflow.start_run(run_name="dense32-l2") as run:
    mlflow.log_params({"units": 32, "l2": 1e-4, "lr": 3e-4, "batch": 512, "seed":
     42})
    mlflow.set_tags({"git_sha": GIT_SHA, "data_sha256": DATA_SHA, "owner": "jserrano
     "})

    hist = model.fit(ds_train, validation_data=ds_val, epochs=30, verbose=0)
    for epoch, auc in enumerate(hist.history["val_auc"]):
        mlflow.log_metric("val_auc", auc, step=epoch)           # a curve, not a
     number
```

```
    mlflow.log_metrics({"val_cost_kes": cost_at(t_star), "p99_latency_ms": p99})
    mlflow.log_artifact("reports/calibration.png")
```

Listing 10.11: Explicit experiment logging with MLflow.

MLflow can also log automatically. For Keras 3 models, the `autolog` function of the `mlflow.keras` module captures the arguments of `fit`, the optimizer configuration, per-epoch metrics and a model summary; for older `tf.keras` code the equivalent lives in the `mlflow.tensorflow` module. Autologging is convenient and genuinely useful for exploratory work.

Pitfall

Autologging is not lineage Autologging sees what passes through `fit`. It cannot know which data snapshot you loaded, which commit you ran, which threshold you intend to deploy or what a missed default costs. A run with a beautiful autologged dashboard and no data hash cannot be reproduced, which makes it useless to an auditor. Always add the lineage tags and the business metric by hand.

10.4.1.3 Versioning the data

A content hash turns "the September data" into an identity. Training reads an immutable snapshot, typically a Parquet file frozen from the live tables; its SHA-256 digest is computed and stored as a run tag (Listing 10.12). Two runs with the same digest used byte-identical data; two runs with different digests did not, whatever their file names say. Tools such as DVC do the same at scale, storing snapshots in the object store and retrieving them by hash (Iterative, 2024).

```
import hashlib

def file_sha256(path, chunk=1 << 20):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        while block := f.read(chunk):
            h.update(block)
    return h.hexdigest()

DATA_SHA = file_sha256(SNAPSHOT)
mlflow.log_input(mlflow.data.from_pandas(train_df, source=SNAPSHOT, name="train"))
```

Listing 10.12: Hashing the training snapshot and recording it in the run.

10.4.1.4 Logging the model with its signature

When the model itself is logged, it should carry a *signature*: the schema of its inputs and outputs, inferred from an example. The signature is the Section 10.3.3.2 contract stored with the weights, and MLflow enforces it when

the model is loaded for inference, so a request with a missing or misnamed column fails loudly instead of silently shifting features (Listing 10.13). The threshold and cost matrix travel as metadata, completing the artefact of Definition 10.1.

```
from mlflow.models import infer_signature

example = {k: v[:5] for k, v in X_val.items()}
signature = infer_signature(example, model(example).numpy())

info = mlflow.keras.log_model(
    model, name="model",                    # MLflow 3; artifact_path= in 2.x
    signature=signature, input_example=example,
    metadata={"threshold": 0.074, "cost_fn_kes": 10000, "cost_fp_kes": 800})
print(info.model_uri)                       # handle used by the registry
```

Listing 10.13: Logging the Keras model with a signature, an input example and the decision metadata.

10.4.1.5 Sweeps and the choice of candidate

Hyperparameter search, from Chapter 4, produces dozens of runs at once. MLflow organises them as *nested* runs: one parent run per sweep, one child per trial, so that the sweep can be compared as a whole and its trials inspected individually. Opening each Optuna trial's run with `nested=True`, inside the sweep's parent run, is all it takes.

Choosing among the results is a multi-objective decision, not a leaderboard. Plot every run on the axes that the product cares about, here validation AUC against p_{99} latency (Figure 10.23), and look at the *Pareto front*, the runs that no other run beats on both axes at once. The run with the highest AUC gains 0.006 over the chosen one at four times the latency, which the budget of Figure 10.11 cannot afford. The decision is made with the budget in hand.

10.4.2 Tracing language-model agents

The agents and retrieval systems of Chapter 9 need the same discipline, adapted to three differences. First, their "parameters" include prompts, the model name, decoding settings and tool schemas, all of which must be logged and versioned exactly like hyperparameters. Second, a single decision is not one prediction but a tree of calls: plan, retrieve, call a tool, draft. MLflow's tracing records that tree as nested spans with timings, inputs and outputs (Figure 10.24), and decorating a function with `@mlflow.trace` is enough to include it. Third, correctness is judged on a *golden set* of cases with known good behaviour, such as the right tool called with the right fields, rather than on a labelled test set in the usual sense.

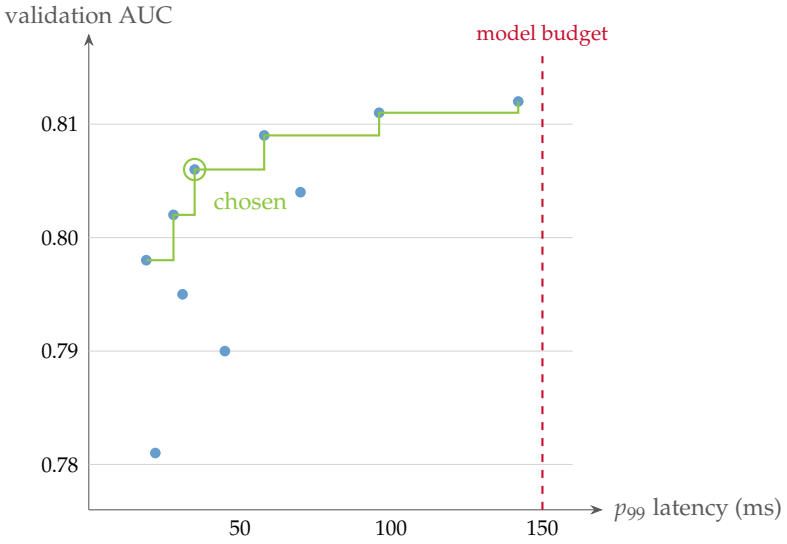


Figure 10.23: Candidate selection on the Pareto front of accuracy and latency (simulated runs).

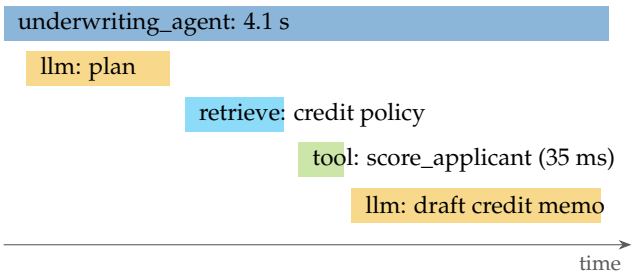


Figure 10.24: A traced agent decision. The deterministic scoring tool is a thin sliver of the total time.

10.4.3 The model registry

Tracking records every experiment. A *registry* records the much smaller set of models that are candidates for production, and governs how they get there.

Definition 10.4 (Registered model, version and alias)**.** A *registered model* is a named entry in the registry, such as `thinfile-scorer`. Each registration creates an immutable, numbered *version* linked to the run that produced it. An *alias* is a movable, named pointer to one version, such as `@champion` for the model in production and `@challenger` for the candidate being evaluated.

The combination is powerful because it separates identity from role (Figure 10.25). Versions never change, so "version 4" means the same bytes forever and always points back to its run, its data hash and its commit. Roles change constantly, and an alias move is how they change. Serving code loads the model through a URI that names an alias,

`models:/thinfile-scorer@champion`,

and never mentions a version number, so *promotion is a pointer move, and so is rollback*: repointing `@champion` from version 5 back to version 4 is one API call that rebuilds nothing. Older MLflow releases used fixed stages (Staging, Production, Archived) for the same purpose; they are deprecated in favour of aliases, which are free-form and can express patterns such as `@shadow`.

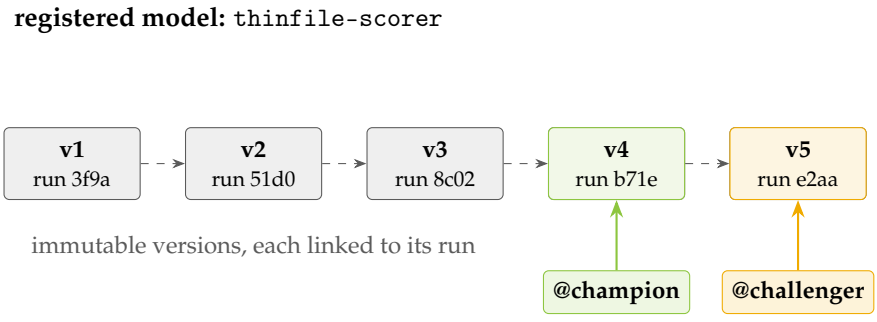


Figure 10.25: Versions are identities; aliases are roles. Serving code only ever names an alias.

```
from mlflow import MlflowClient
client, NAME = MlflowClient(), "thinfile-scorer"

mv = mlflow.register_model(info.model_uri, NAME,
                           tags={"git_sha": GIT_SHA, "data_sha256": DATA_SHA})
client.set_registered_model_alias(NAME, "challenger", mv.version)
```

```
# ... the gates pass and the risk officer approves ...
champ = client.get_model_version_by_alias(NAME, "champion")
client.set_model_version_tag(NAME, champ.version, "previous_champion", "true")
client.set_registered_model_alias(NAME, "champion", mv.version)    # promotion
```

Listing 10.14: Registration and promotion in code; this is what the pipeline calls.

10.4.3.1 Bake the model in, or fetch it by alias?

The container built in Section 10.6.1 bakes the model into the image. The registry offers an alternative: an image that contains no model, whose lifespan handler loads `models:/thinfile-scorer@champion` at startup. The fetch design promotes a model without rebuilding anything, but it makes startup depend on the registry being reachable and weakens the statement "this image digest made this decision". The bake design keeps that statement true and makes audit trivial, at the cost of a rebuild per model release. For a regulated lender the sensible default is to *bake in*, and to let the delivery pipeline of Section 10.6.2 rebuild the image automatically whenever `@champion` moves. Both paths remain auditable; the bake path is simply auditable with less effort.

10.4.3.2 A human in the loop

Automation produces evidence; a person remains accountable for the decision to use it. Figure 10.26 shows PesaCred's promotion workflow. The training pipeline registers a new version as `@challenger`. Automated gates, described in the next section, test its cost, fairness and behaviour; a failure stops the process. If every gate passes, a named risk officer reviews the generated report and approves, and only then does `@champion` move.

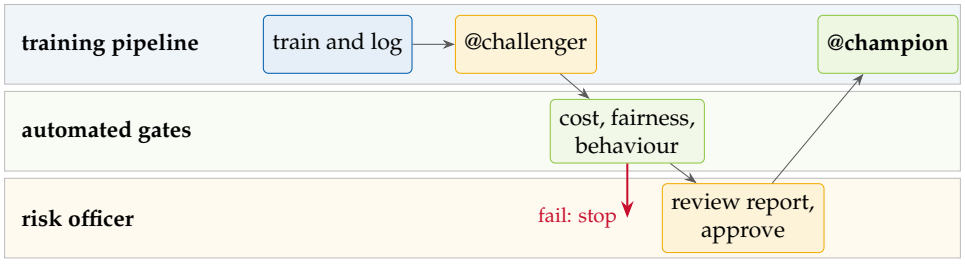


Figure 10.26: Automated evidence, accountable sign-off. The approval is itself a recorded event.

Kenya Context

The approval step matches CBK expectations that credit decisions and the models behind them are governed by accountable people, and it implements a Section 35 safeguard at the level of the model rather than of the individual decision. Record who approved, when, and on the basis of which report; in GitHub Actions this is a protected environment with required reviewers (Section 10.6.3.3), and the approval is written to the audit log automatically.

10.5 Testing machine learning systems

10.5.1 Why ML testing is different

Ordinary software testing checks code against a specification. A machine learning system can break in three places, and only one of them is code (Figure 10.27). *Code* breaks when a refactor changes the preprocessing. *Data* breaks without any commit at all, when an upstream team renames a column or a mobile network changes its message format. And the *model* breaks statistically: a two-point drop in recall on the betting category raises no exception. Breck et al. (2017) turned this observation into a rubric, the ML Test Score, which grades a system's tests across all three.

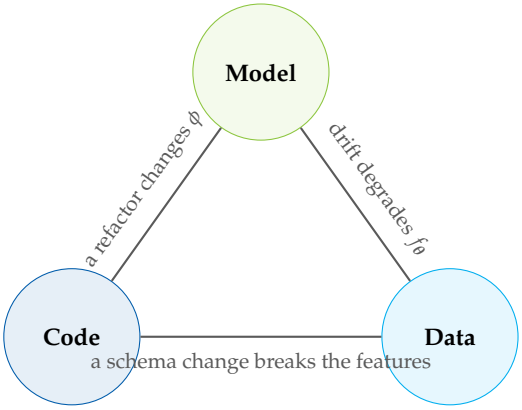


Figure 10.27: Three things can break, and each needs its own tests.

The tests are organised as a pyramid (Figure 10.28): many fast, cheap tests at the base that run on every commit, and few slow, expensive ones at the top that run only when a model is a candidate for promotion. A failure low in the pyramid is cheap to diagnose; a failure at the top usually means something lower down should have caught it.

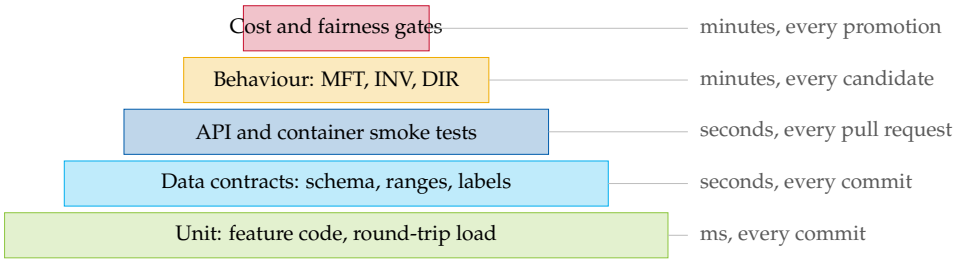


Figure 10.28: A testing pyramid for machine learning systems.

10.5.2 Unit tests and data contracts

The base of the pyramid is ordinary `pytest` applied to feature code, plus the round-trip assertion of Listing 10.2. The one habit worth stressing is to write a test for every bug found: if reversed transactions once inflated the inflow feature, a three-line test asserting that a reversal cancels rather than counts keeps that bug dead.

Data needs tests of its own, because it changes without a commit. A *data contract* is an explicit, checked agreement between the team that produces a table and the team that consumes it: column names and types, value ranges, allowed categories, and plausible label rates. Libraries such as pandera express it in a few lines (Bantilan, 2024), and validating every snapshot against it before training turns a silent upstream change into a loud, early failure (Listing 10.15).

```
import pandera.pandas as pa
from pandera import Check

schema = pa.DataFrameSchema({
    "tenure_months": pa.Column(int, Check.in_range(0, 480)),
    "avg_monthly_inflow_kes": pa.Column(float, Check.ge(0)),
    "txn_count_90d": pa.Column(int, Check.ge(0)),
    "region": pa.Column(str, Check.isin(REGIONS)),
    "defaulted": pa.Column(int, Check.isin([0, 1])),
}, strict=True)                                  # unexpected columns fail too

def test_snapshot_contract(snapshot):
    schema.validate(snapshot, lazy=True)         # report every violation at once
    assert snapshot["defaulted"].mean() < 0.15   # label sanity: default rate
```

Listing 10.15: A data contract for the training snapshot.

10.5.3 Behavioural tests

Aggregate metrics say how often a model is right, not *what* it has learned. Behavioural testing, systematised as CheckList by Ribeiro et al. (2020), checks specific capabilities that a domain expert would sign off on. Three test types cover most needs (Figure 10.29).

Definition 10.5 (Behavioural test types)**.** Let f be the model's score and δ its decision.

- A *minimum functionality test* (MFT) requires a fixed outcome on a curated set: $\delta(\boldsymbol{x}) = y^\star$ for all $\boldsymbol{x} \in \mathcal{S}$.
- An *invariance test* (INV) requires that a perturbation τ that should be irrelevant leaves the score essentially unchanged: $|f(\boldsymbol{x}) - f(\tau(\boldsymbol{x}))| \leq \epsilon$.
- A *directional expectation test* (DIR) requires that a perturbation moves the score in a known direction: $f(\boldsymbol{x} + \Delta \boldsymbol{e}_j) \leq f(\boldsymbol{x})$ for a feature j that should only lower risk.

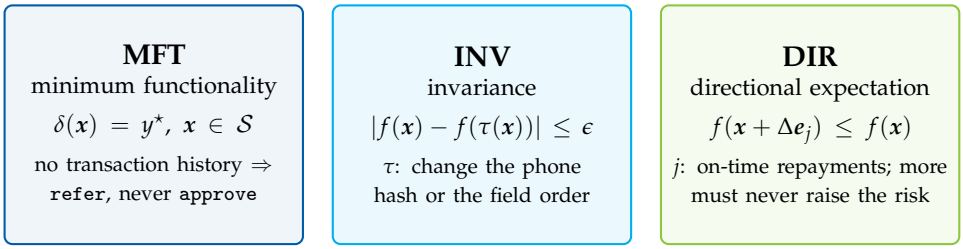


Figure 10.29: Three behavioural test types for the credit scorer.

Each test encodes a belief that a credit officer would sign, which is what makes a failure useful: it is the start of a conversation about the model, not merely a red cross. Because models are statistical, directional tests are usually written with a budget, as in Listing 10.16, which allows at most 1% of probe rows to violate monotonicity. The tolerance and the budget are policy decisions; record who set them and why, exactly as the cost matrix is recorded.

```
import numpy as np, pytest

def pd_of(model, rows):
    return model({k: v[:, None] for k, v in rows.items()}, training=False).numpy().
     ravel()

@pytest.mark.parametrize("delta", [1, 3, 6])
def test_dir_more_ontime_repayments_never_raises_pd(challenger, probe, delta):
    bumped = {**probe, "ontime_repayments_12m": probe["ontime_repayments_12m"] +
     delta}
    violations = pd_of(challenger, bumped) > pd_of(challenger, probe) + 1e-4
    assert violations.mean() <= 0.01, f"{violations.mean():.1%} rows violate
     monotonicity"
```

Listing 10.16: A directional test with a violation budget, run against the challenger.

10.5.4 Statistically honest gates

At the top of the pyramid sit the gates that decide promotion. The first compares the challenger with the champion on the business metric. Doing that naively, by comparing two expected costs, ignores the sampling noise in the holdout, and a gate that ignores noise will promote lucky models and block good ones at random.

The honest version exploits *pairing*. Both models score the same frozen holdout, so for each applicant i we can compute the difference in cost, $d_i = C_{\text{chal}}(i) - C_{\text{champ}}(i)$, and its mean $\Delta C = \bar{d}$. Pairing removes the variation between applicants, which is by far the largest source of noise. The bootstrap of Chapter 3 then gives an interval: resample the rows B times, recompute $\Delta C^{(b)}$ each time, and

$$\text{promote only if} \quad q_{0.975}\big(\Delta C^{(b)}\big) \;<\; 0, \tag{10.9}$$

that is, only if even the pessimistic end of the interval shows a saving (Figure 10.30). In the illustrated case the challenger saves KES 9 per applicant, with a 95% interval of $[-15, -3]$, and passes. Exercise 10.7 shows how the size of the holdout decides whether a real saving of this magnitude can pass at all.

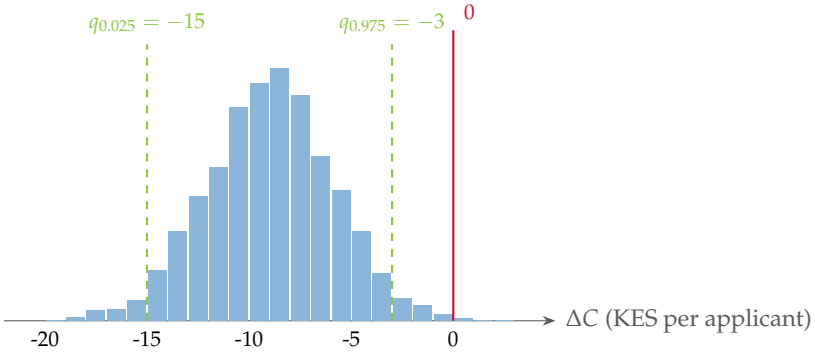


Figure 10.30: A paired bootstrap of the cost difference (simulated). The whole interval lies below zero, so the challenger is credibly cheaper.

The second gate concerns fairness, and aggregate metrics are exactly what hides unfairness. Compute the approval rate among borrowers who repaid, the equal-opportunity true positive rate of Chapter 3, separately for each region g, and require the gap between the best- and worst-served regions to stay below a policy threshold,

$$\Gamma \;=\; \max_g \mathrm{TPR}_g - \min_g \mathrm{TPR}_g \;\le\; \tau. \tag{10.10}$$

In Figure 10.31 the challenger serves good borrowers in North Eastern noticeably worse than those in Nairobi, $\Gamma = 0.12 > \tau = 0.10$, and the gate

fails, whatever the aggregate AUC says. Two refinements matter in practice. Regions with few loans give noisy rates, so the gate should use a confidence bound rather than a point estimate (Exercise 10.8). And sensitive attributes used for such audits must be held separately from the model's features, used for evaluation only, and access-controlled under the Data Protection Act.

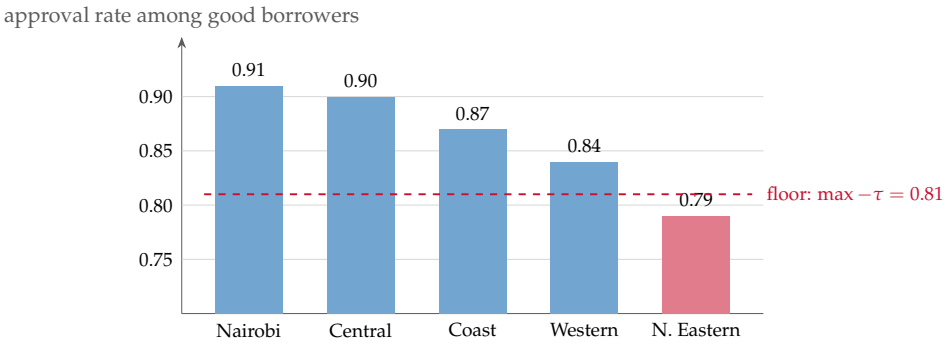


Figure 10.31: A fairness slice gate. The aggregate hides a region the challenger serves worse.

The gates themselves are ordinary `pytest` functions that read their baselines from the registry (Listing 10.17). One detail protects the whole construction: the test asserts that the holdout file's hash equals the hash recorded when the champion was evaluated, so that challenger and champion are always measured with the same yardstick.

```
def test_cost_gate(challenger, champion, holdout):
    d = paired_cost_diff(challenger, champion, holdout, t_star=0.074)   # KES per
     row
    boot = [d[idx].mean() for idx in bootstrap_indices(len(d), B=2000, seed=7)]
    assert np.quantile(boot, 0.975) < 0, "challenger not credibly cheaper"

def test_fairness_gate(challenger, holdout):
    tpr = approval_rate_among_good(challenger, holdout, by="region", t_star=0.074)
    assert tpr.max() - tpr.min() <= 0.10, tpr.to_dict()

def test_latency_gate(challenger, probe):
    assert p99_latency_ms(challenger, probe, n=2000) <= 150          # latency
     budget

def test_holdout_is_the_frozen_one(holdout_path):
    champ = MlflowClient().get_model_version_by_alias("thinfile-scorer", "champion")
    assert file_sha256(holdout_path) == champ.tags["holdout_sha256"]
```

Listing 10.17: Promotion gates as tests, with baselines and the holdout identity taken from the registry.

Pitfall

Flaky gates get switched off A gate that fails on some runs and passes on others, with the same code and data, destroys trust faster than having no gate, because the team learns to re-run until it passes and then to disable it. Freeze the holdout and pin its hash; fix the seeds of every sampling step, including the bootstrap; gate on confidence bounds rather than point estimates; and separate *fail*, which blocks, from *warn*, which only reports.

10.5.5 Gating language-model agents

Golden sets are small, typically one or two hundred cases, and small samples make point estimates optimistic. A CI gate for an agent should therefore use a lower confidence bound on the pass rate. With k passes out of n cases, $\hat{p} = k/n$, the Wilson lower bound at confidence level z is

$$\mathrm{LB} = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \tag{10.11}$$

which behaves sensibly even for pass rates near one, where the textbook normal approximation does not. With 184 passes out of 200, $\hat{p} = 0.92$ and $\mathrm{LB} = 0.874$, which clears a threshold of 0.85. The same gate checks cost per case and p_{95} latency against their budgets, and, as for the credit model, the grader itself must be pinned: when a language model is used to judge answers, changing the judge changes the scores.

10.6 Shipping: containers, delivery and governance

Everything is now built and checked: an artefact, a service, a tracked and registered model, and a suite of tests and gates. What remains is to move all of it into production repeatedly and safely. That means packaging the service as a container, automating the path from a commit to a deployment, and keeping the record that a regulator will ask for.

10.6.1 Containerising with Docker

10.6.1.1 The environment is part of the model

Section 10.2.5 pinned the Python dependencies. A service also depends on the operating system libraries beneath them, on the Python interpreter itself, and on configuration. A *container* packages all of these with the application into a single image that runs identically on a laptop, a continuous

integration runner and a server in a Nairobi data centre (Docker Inc., 2024). Unlike a virtual machine, a container does not carry a guest operating system; it shares the host's kernel and isolates only what differs (Figure 10.32). Images are therefore megabytes rather than gigabytes and start in seconds rather than minutes.

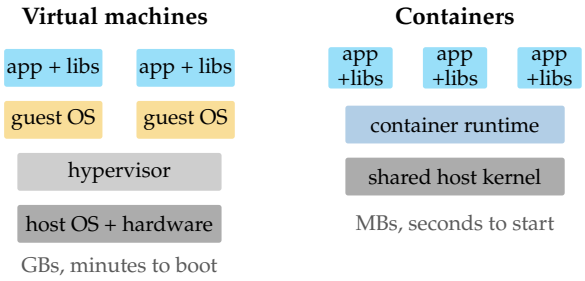


Figure 10.32: Virtual machines virtualise hardware; containers virtualise the operating system.

Three nouns organise the rest (Figure 10.33). A `Dockerfile` is the recipe. `docker build` turns it into an *image*, a read-only stack of layers. The image is pushed to a *registry*, such as the GitHub Container Registry or a self-hosted Harbor, and pulled by any server that runs it as a *container*, a live instance with its own thin writable layer. Containers are replaced, never patched in place: to change anything, you build a new image.

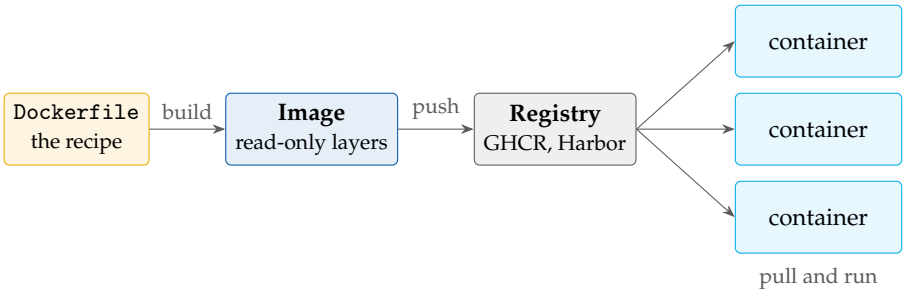


Figure 10.33: Build once, push once, run anywhere, many times.

10.6.1.2 Layers and the build cache

Each instruction in a Dockerfile creates a layer, and Docker caches layers: if an instruction and everything before it are unchanged, the cached layer is reused. A change invalidates that layer *and every layer above it*. The practical consequence is an ordering rule: put what changes rarely at the bottom and what changes often at the top (Figure 10.34). Copy the lock file and install dependencies first; copy the application code last. With the order reversed, a one-line edit to the endpoint re-downloads TensorFlow.

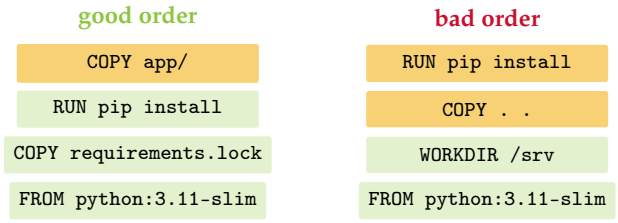


Figure 10.34: Layer ordering decides how much a small change costs to rebuild.

A `.dockerignore` file does for the build context what the ordering rule does for the cache. It keeps `data/`, `.git` and, above all, `.env` files with credentials out of the image. Anything copied into a layer stays in the image's history even if a later layer deletes it.

10.6.1.3 A multi-stage Dockerfile

Installing Python packages can require compilers and headers that the running service does not need. A *multi-stage* build installs dependencies into a virtual environment in a first, disposable stage, and copies only that environment into a clean second stage (Listing 10.18). The second stage also applies the hardening that a credit-decision service warrants: it runs as a non-root user, carries a healthcheck, and sets the thread counts from Section 10.3.3.5 explicitly.

```
# ---------- stage 1: builder (discarded) ----------
FROM python:3.11-slim AS builder
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.lock .
RUN pip install --no-cache-dir --require-hashes -r requirements.lock

# ---------- stage 2: runtime (shipped) ----------
FROM python:3.11-slim
RUN useradd --create-home --uid 10001 app
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH" PYTHONUNBUFFERED=1 \
    OMP_NUM_THREADS=1 TF_NUM_INTRAOP_THREADS=1
WORKDIR /srv
COPY app/ app/
COPY artefacts/ artefacts/
USER app
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8000/health
    ')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers",
    "2"]
```

Listing 10.18: A two-stage Dockerfile for the scoring service.

The model is baked into this image, which means the image digest *is* the release: whoever holds the digest holds exactly the code, dependencies and model that made a given decision. Section 10.4.3.1 weighed that against fetching the model from the registry at startup.

10.6.1.4 Size, security and tags

Image size is not vanity. The time to pull an image over a link of bandwidth B is $t = S/B$, so a 2.1 GB image takes about 28 minutes over a 10 Mbit/s link and a 0.9 GB image about 12 minutes (Figure 10.35). Because layers are cached on the receiving side as well, a well-ordered image only ships its top layers on an ordinary release, often a few tens of megabytes.

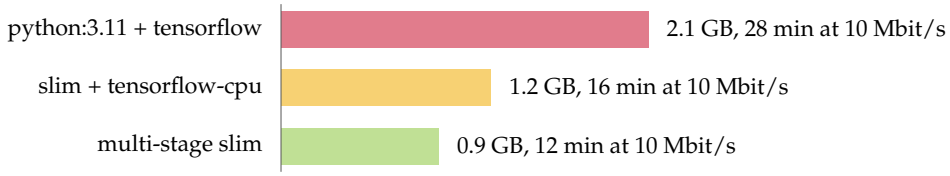


Figure 10.35: Image size and pull time on a constrained link (illustrative sizes).

Kenya Context

Branch servers, agent-network hubs and county data centres often pull images over shared or metered links. Small images, stable base layers that are rarely rebuilt, and a local registry mirror in Nairobi cut both deployment time and data cost, and they make a rollback at 2 a.m. a matter of seconds rather than of waiting for a download.

Security hardening follows the principle of defence in depth. Use a **minimal base** image, since every package you do not ship is a vulnerability you do not have. Run as a **non-root user**, so that a compromised process does not own the host. Keep **secrets out of images**, because layers are permanent; inject them at run time. **Scan** every build for known vulnerabilities, for example with Trivy, and fail the build on critical findings. And **pin the base image by digest**, not only by tag.

Finally, a tag should tell you what is inside. A reference such as

```
ghcr.io/ilabafrica/thinfile-scorer:1.3.0-a1b2c3d
```

names the registry, the repository, the model version and the git commit. The tag `latest`, by contrast, is a moving pointer rather than a version, and a production deployment that references it cannot say what it is running.

10.6.2 Continuous integration, delivery and training

Everything so far can be run by hand. Automation removes the hand, and it comes in three loops distinguished by their triggers (Figure 10.36).

Definition 10.6 (CI, CD and CT)**.** *Continuous integration* (CI) runs tests on every proposed change to the code, before it is merged. *Continuous delivery* (CD) builds, scans and deploys every merged change through a sequence of environments. *Continuous training* (CT) retrains, evaluates and registers a model on a schedule or when monitoring detects drift.

CI and CD come from software engineering. CT is what makes the discipline MLOps rather than DevOps: in a machine learning system, new *data* can trigger a release, even when nobody has changed a line of code.

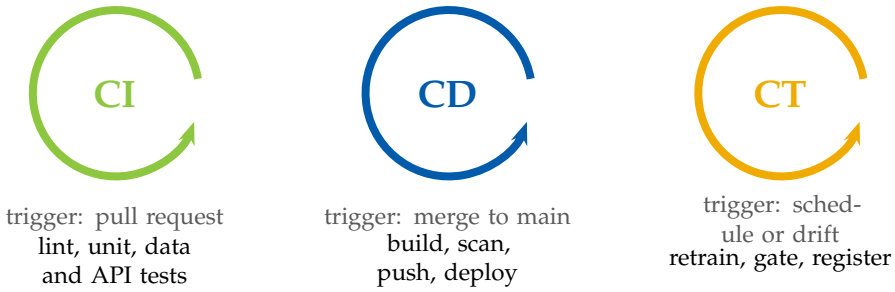


Figure 10.36: Three automation loops with three different triggers.

GitHub Actions, the automation service built into GitHub, expresses these loops as *workflows*: YAML files in `.github/workflows/` that declare triggers, jobs and steps (GitHub, 2024). Jobs run in parallel on *runners* unless linked by `needs:`; steps inside a job run a shell command or a reusable action; and *environments* add approval rules and scope secrets to particular stages (Figure 10.37).

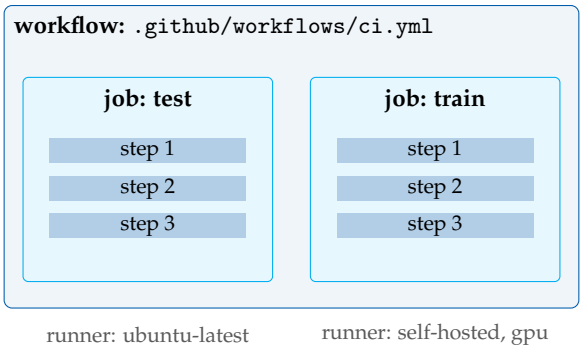


Figure 10.37: Anatomy of a GitHub Actions workflow.

10.6.3 The three workflows

PesaCred's repository holds three workflow files, one per loop. Each is short, because every step it runs is a command that already works on a laptop.

10.6.3.1 Workflow 1: continuous integration

The CI workflow runs on every pull request against the main branch (Listing 10.19). It installs the locked dependencies, lints and type-checks the code, runs the unit, data and API tests, and builds the image and smoke-tests it. Configured as a *required status check* in the repository's branch protection, it makes the Session 10 tests the gate that every change must pass before merging.

```yaml
name: ci
on:
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: pip
      - run: pip install --require-hashes -r requirements.lock
      - run: pip install -r requirements-dev.txt
      - run: ruff check . && mypy app
      - run: pytest tests/unit tests/data tests/api -q
      - run: docker build -t scorer:ci . && ./scripts/smoke.sh scorer:ci
```

Listing 10.19: `ci.yml`: tests on every pull request.

10.6.3.2 Workflow 2: continuous training

The CT workflow retrains on a schedule, and can be triggered manually or, in Chapter 11, by a drift alert (Listing 10.20). It runs on a *self-hosted* runner, a machine PesaCred owns in Nairobi with a GPU and direct access to the data, so that training data never leaves the country and training minutes are not billed. Its steps are the lower half of the promotion workflow: pull the hashed snapshot, validate the data contract, train and log, run the behavioural tests and gates, and register the result as `@challenger`. Any failing step stops the job, so nothing that fails a gate is ever registered.

```yaml
name: train-and-validate
on:
  workflow_dispatch:
  schedule:
    - cron: "0 2 * * 1"                # Mondays 02:00 UTC = 05:00 EAT
```

```yaml
jobs:
  train:
    runs-on: [self-hosted, gpu]
    env:
      MLFLOW_TRACKING_URI: ${{ vars.MLFLOW_URI }}
    steps:
      - uses: actions/checkout@v4
      - run: dvc pull data/snapshots/latest.parquet
      - run: pytest tests/data -q
      - run: python -m pipeline.train --config configs/scorer.yaml
      - run: pytest tests/model -q       # behaviour and gates
      - run: python -m pipeline.register --alias challenger
```

Listing 10.20: `train.yml`: scheduled retraining on a self-hosted runner.

10.6.3.3 Workflow 3: continuous delivery

The CD workflow runs on every merge to main and whenever `@champion` moves (Listing 10.21). It fetches the champion model, builds the image, scans it and refuses to continue on critical vulnerabilities, pushes it to the registry tagged with the commit, deploys it to a staging environment and smoke-tests it there. The production deployment job references a GitHub *environment* named `production` configured with required reviewers, which is where the risk officer's approval of Section 10.4.3.2 lives. Production credentials exist only inside that environment, so even a compromised earlier job cannot deploy.

```yaml
name: cd
on:
  push:
    branches: [main]
env:
  IMAGE: ghcr.io/ilabafrica/thinfile-scorer:${{ github.sha }}
jobs:
  build:
    runs-on: ubuntu-latest
    permissions: {contents: read, packages: write}
    steps:
      - uses: actions/checkout@v4
      - run: python -m pipeline.fetch_model --alias champion
      - run: docker build -t "$IMAGE" .
      - run: ./scripts/trivy_scan.sh "$IMAGE"          # exit 1 on critical CVEs
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - run: docker push "$IMAGE"
  deploy-staging:
    needs: build
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - run: ./deploy.sh staging "$IMAGE" && ./scripts/smoke.sh https://staging.
    scorer
  deploy-production:
```

```
    needs: deploy-staging
    runs-on: [self-hosted, nairobi-dc]
    environment: production                              # required reviewers
    steps:
      - run: ./deploy.sh production "$IMAGE" --canary 5
```

Listing 10.21: `cd.yml`: build, scan, push, then deploy through protected environments.

Secrets deserve a sentence of their own. Static credentials stored in repository secrets remain valid for years, and a leak is permanent. Where the target supports it, prefer OpenID Connect federation, in which GitHub issues a short-lived token that the cloud or registry exchanges for a credential valid for minutes and scoped to one repository, branch and environment. Never commit secrets, never bake them into images, and never print them in logs.

10.6.4 Deployment strategies and the whole pipeline

The last line of Listing 10.21 deploys a *canary*. Three strategies are common (Figure 10.38). In **shadow** deployment the new model scores every request in parallel with the old one but its decisions are only logged, never acted on. In a **canary** deployment the new model makes real decisions for a small slice of traffic, say 5%, which grows as confidence grows. In **blue–green** deployment the new version runs beside the old, and traffic switches all at once, with an equally instant switch back.

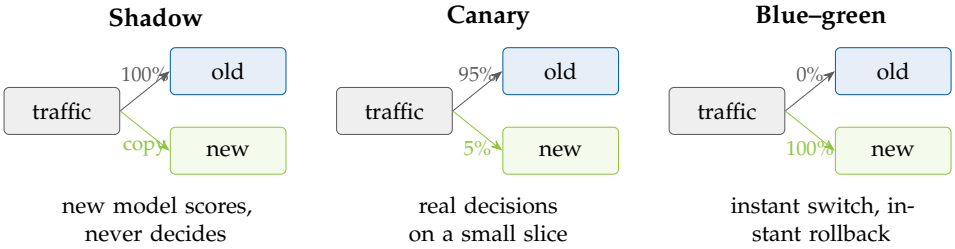


Figure 10.38: Three deployment strategies. For credit, shadow comes first.

Credit adds a twist that most software deployment advice ignores: the label arrives late. Whether a loan defaults is known thirty days or more after disbursement, so a canary cannot be judged on defaults within any reasonable window. It is judged on *proxies* instead: the approval rate, the distribution of scores, latency and error rates, all of which are available immediately. Shadow deployment is the natural first step for a credit model, since it allows the new model's decisions to be compared with the old one's on every live request without exposing any borrower to them. And rollback, whichever strategy is used, is the alias move of Section 10.4.3 followed by an automatic redeploy.

Figure 10.39 assembles the whole pipeline. Every arrow leaves a record: a commit, a run, a registered version, an approval, an image digest, a deployment. That observation is the subject of Section 10.6.5.

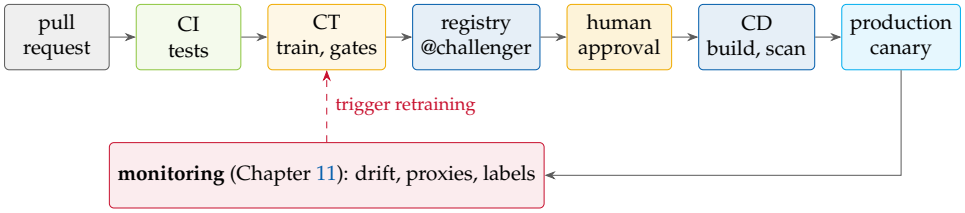


Figure 10.39: The complete pipeline, from a pull request to a monitored production model and back.

10.6.5 Auditability and constrained infrastructure

The pipeline of Figure 10.39 was built for engineering reasons, and it turns out to be most of a compliance programme. Consider the question a CBK examiner or a complaining borrower actually asks: why was this application declined? Starting from the reference in the SMS the borrower received, the decision log gives the `request_id`, the probability, the threshold, the reason codes and the model version; the registry maps the version to its run; the run gives the data hash, the parameters and the commit; and the commit gives the code (Figure 10.40). Five hops, every one of them recorded automatically.

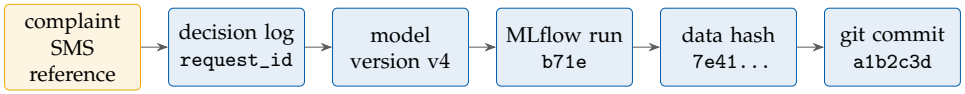


Figure 10.40: From a customer complaint to the exact code and data, in five recorded hops.

Kenya Context

The CBK Digital Credit Providers Regulations 2022 require complaint handling that is only possible if decisions can be reconstructed, and supervisory reviews ask the same questions at scale. The Data Protection Act 2019 gives borrowers rights of access and of objection to solely automated decisions, which are answerable from the same chain. Two design choices from this chapter make the chain lawful as well as useful: the decision log stores a pseudonymous key rather than the phone number (Section 10.3.3.2), and both the tracking server and the object store sit in-country (Section 10.4.1.1). No separate compliance project is needed; the pipeline produces the evidence as a by-product.

Most of the stack is also cheap to run. A single well-provisioned server in Nairobi can host a self-hosted GitHub runner with a GPU, the MLflow server with PostgreSQL, an S3-compatible object store for artefacts and DVC snapshots, and a registry mirror with a package cache. Self-hosted runners avoid per-minute billing for training and keep data in the country. Caching dependencies, Docker layers and base images, and keeping base layers stable, means that an ordinary release crosses the uplink as a few tens of megabytes. Heavy jobs are scheduled off-peak, which is why the CT workflow runs at five in the morning, Nairobi time.

Pitfall

Automation that hides risk Each of the following makes a pipeline look mature while hiding the risks it exists to control: comparing single runs instead of seed distributions; autologging without a data hash or commit; gating on point estimates, or on a holdout that silently changed; secrets in YAML, images or logs; loading the model per request or running CPU-bound inference in `async def`; failing open when the service times out; and promoting to production automatically, with no human approval and no rehearsed rollback.

10.7 Conclusions

The deployable unit of a machine learning system is a decision function, not a file of weights: preprocessing, threshold, contract and lineage travel with the parameters, and changing any of them creates a new version. Putting preprocessing inside the model graph removes training–serving skew by construction, and the threshold follows from the cost matrix, $t^\star = C_{FP}/(C_{FP} + C_{FN})$, provided the probabilities are calibrated.

Serving begins with a pattern and a budget. Most computation belongs outside the request path; latency is budgeted by component and stated as a percentile, because fan-out makes the system's tail worse than any component's; and Little's law sizes the workers. FastAPI turns a typed contract into validation, documentation and a tool schema for agents; CPU-bound inference goes in `def` endpoints, loaded once at startup; and the response is an audit record, because a credit decision is answerable to the borrower who received it.

Reproducibility means recording all five inputs of training: code, data, configuration, environment and seed. MLflow records them as runs, a registry turns the good runs into governed versions, and aliases make promotion and rollback single pointer moves. Tests span code, data and model, and the gates at the top of the pyramid compare challenger and champion with paired, bootstrapped business cost and with fairness across regions, on a frozen holdout.

Shipping brings the three together. Docker packages the service with its environment into a small, non-root, scanned image whose digest is the release; GitHub Actions automates the three loops of continuous integration, delivery and training; a protected environment keeps the risk officer's approval in the path; and shadow and canary deployments cope with labels that arrive a month late. The audit trail that Kenyan regulation requires is then not a separate project but a by-product of the pipeline.

It is worth carrying away a short list of what "ready for production" means, since each item is a section of this chapter. The artefact carries its own preprocessing, threshold and lineage (Section 10.2). The service has a typed contract, a latency budget stated in percentiles, health and readiness probes, and it fails closed (Section 10.3). Every model traces back to a commit, a data hash and a run, and reaches production only through an alias that a named person moved (Section 10.4). Data, behaviour, cost and fairness are tested on a frozen holdout, with gates that survive being rerun (Section 10.5). And the path from a commit to a deployment, and back out again through a rollback, is automated and rehearsed (Section 10.6). A system missing any one of these can still score well offline; it cannot be run.

Chapter 11 closes the loop: it watches the deployed model for drift and gives the continuous-training workflow of this chapter its trigger.

Exercises

1. **The threshold is part of the model.** PesaCred's costs are $C_{FN} =$ KES 10,000 and $C_{FP} =$ KES 800. (a) Compute $t^\star$. (b) On a validation set of 10,000 applicants containing 400 defaulters, threshold 0.5 gives 260 false negatives and 150 false positives, while $t^\star$ gives 90 false negatives and 1,900 false positives. Compute the expected cost per applicant under each. (c) The business now estimates that a wrongly declined borrower costs KES 1,500. Recompute $t^\star$, recompute both costs from part (b), and explain what the result implies for how threshold changes should be versioned.

2. **Budgets and tails.** (a) A request calls five independent services, each with a p_{99} target. What fraction of requests hit at least one component's p_{99}? Repeat for eight services. (b) What per-component percentile q would be needed for 99% of five-call requests to see no slow call? (c) Using Figure 10.11, explain why the 900 ms of headroom is not waste.

3. **Sizing for month-end.** Mean service time is $\bar{S} = 25$ ms and the utilisation ceiling is $\rho_{\max} = 0.6$. (a) Compute the offered load and the number of workers at $\lambda = 120$ requests per second. (b) Repeat for the month-end peak of $\lambda = 300$. (c) Each worker holds its own copy of a

350 MB model; how much memory does the peak configuration need for models alone? (d) If the mean time in the system at peak is 40 ms, how many requests are in flight on average?

4. **Why batch endpoints exist.** A model call costs $T(b) = 8 + 0.15\,b$ milliseconds for a batch of b rows. (a) Compute the cost per row for $b = 1$, 32 and 256. (b) The nightly job rescores 3 million applicants. How long does it take through the online endpoint ($b = 1$) and through a batch endpoint ($b = 256$), ignoring network time? (c) Why must the online path still use $b = 1$?

5. **Image size on a real link.** (a) Compute the pull time at 10 Mbit/s for images of 2.1 GB and 0.9 GB. (b) A release changes only the application layer, 40 MB, on a well-ordered image already cached at the branch. How long is the pull? (c) The same release on an image whose Dockerfile copies the code before installing dependencies: which layers are shipped, and why?

6. **Seed variance.** Five seeds of configuration A give validation AUCs 0.806, 0.814, 0.813, 0.803, 0.804; five seeds of configuration B give 0.803, 0.797, 0.815, 0.805, 0.794. (a) Compute each mean, standard deviation and 95% confidence interval using $t_{4,\,0.975} = 2.776$. (b) A colleague reports that B is better because its best seed beats A's worst. Evaluate the claim. (c) What would you change in the experiment before drawing any conclusion?

7. **How big must the holdout be?** The per-applicant cost difference between challenger and champion has mean -9 KES and standard deviation 310 KES. Use the normal approximation to the paired interval, $\bar{d} \pm 1.96\,s/\sqrt{n}$. (a) Does the gate of Equation (10.9) pass with a holdout of 4,000 applicants? With 10,000? (b) Find the smallest holdout for which a true saving of this size passes. (c) Explain why an unpaired comparison of the two models' mean costs would need a much larger holdout.

8. **Fairness gates on small slices.** In the holdout, 1,820 of 2,000 good borrowers in Nairobi were approved, and 95 of 120 in North Eastern. (a) Compute both rates and the gap Γ; does the gate with $\tau = 0.10$ pass? (b) Compute the 95% Wilson interval for each rate. (c) Argue whether the gate should use point estimates or bounds, and which bound, bearing in mind who bears the cost of each kind of error.

9. **Gating an agent.** An underwriting agent passes 184 of 200 golden cases. (a) Compute $\hat{p}$ and the Wilson lower bound; does it clear $\tau = 0.85$? (b) After a prompt edit it passes 176 of 200. Recompute. (c) A teammate proposes a quicker golden set of 50 cases, on which the

agent passes 46. Compute the lower bound and explain what the comparison with part (a) shows about golden-set size.

10. **A promotion decision.** Challenger v5 reports: cost-gate interval $[-14, -2]$ KES per applicant; fairness gap $\Gamma = 0.08$; p_{99} latency 162 ms; monotonicity violations on 0.6% of probe rows; holdout hash equal to the champion's. (a) State which gates pass and which fail, and whether v5 may become `@champion`. (b) Propose two technical options that could make a model with v5's accuracy deployable, referring to Section 10.3.2. (c) Suppose v5 had been promoted and, a week later, its canary shows the approval rate falling from 62% to 55%. Describe the rollback, the records it produces, and why defaults cannot yet tell you whether v5 was wrong.

Key terms

MLOps; maturity levels; decision function; model artefact; training–serving skew; preprocessing layers; serialisation format; cost-optimal threshold; metadata sidecar; lock file; batch, online, streaming and edge serving; feature store; latency budget; tail latency; percentile; fan-out; Little's law; offered load; utilisation; model-in-service; model server; FastAPI; Pydantic; contract; lifespan handler; event loop; liveness and readiness; fail closed; batch amortisation; container; image; registry; layer cache; multi-stage build; image digest; reproducibility; lineage; seed variance; experiment tracking; run; backend and artefact stores; autologging; data hash; model signature; nested runs; Pareto front; model registry; version; alias; champion and challenger; testing pyramid; data contract; behavioural testing; paired bootstrap; promotion gate; fairness gate; continuous integration, delivery and training; workflow; runner; environment; OpenID Connect; shadow, canary and blue–green deployment; tracing; golden set; Wilson bound; audit chain.

Further reading

The best single companion to this chapter is Huyen (2022), whose chapters on deployment, on experiment tracking and versioning, and on continual learning and testing in production cover the same ground with more industrial examples; read it alongside Sections 10.3 and 10.4.1. Géron (2019) gives the TensorFlow-specific view of saving models, serving them with TF Serving and deploying them to mobile devices, in its chapter on training and deploying models at scale. The case for treating everything around the model as first-class engineering was made by Sculley et al. (2015), a short paper that every data scientist should read once, and Kreuzberger

et al. (2023) provides a careful survey of the definitions, roles and architecture of MLOps that this chapter has used informally. Google Cloud (2020) is the origin of the maturity levels.

On serving performance, Dean and Barroso (2013) is the classic account of why the tail dominates at scale and what can be done about it. On testing, Breck et al. (2017) turns the vague ambition of "testing ML" into a rubric of concrete tests, and Ribeiro et al. (2020) introduces the behavioural test types used in Section 10.5.3; although written for language models, its ideas transfer directly to tabular credit models. Zaharia et al. (2018) describes the design of MLflow. For the tools themselves, the official documentation of FastAPI (Ramírez, 2024), Docker (Docker Inc., 2024), GitHub Actions (GitHub, 2024), DVC (Iterative, 2024) and pandera (Bantilan, 2024) is reliable and current, and preferable to tutorials that age quickly. Finally, the regulatory framing rests on primary sources: the Data Protection Act 2019 (Republic of Kenya, 2019) and the CBK Digital Credit Providers Regulations 2022 (Central Bank of Kenya, 2022).